
Software Requirements Specification

for

Academic Feedback and Improvement Management System

Version 1.0 Approved

Prepared by BRISON B

Department of Computer Science and Engineering

september 2026

Table of Contents

1.	Introduction	2
o	1.1 Purpose	2
o	1.2 Document Conventions	2
o	1.3 Intended Audience and Reading Suggestions	2
o	1.4 Product Scope	2
o	1.5 References	3
2.	Overall Description	4
o	2.1 Product Perspective	4
o	2.2 Product Functions	4
o	2.3 User Classes and Characteristics	4
o	2.4 Operating Environment	12
o	2.5 Design and Implementation Constraints	12
o	2.6 User Documentation	12
o	2.7 Assumptions and Dependencies	13
3.	External Interface Requirements	13
o	3.1 User Interfaces	13
o	3.2 Hardware Interfaces	13
o	3.3 Software Interfaces	14
o	3.4 Communication Interfaces	14
4.	System Features	15
o	4.1 Student Login	15
o	4.2 Feedback Submission	16
o	4.3 Report Generation	17
o	4.4 Faculty Management	17
o	4.5 Student Management	17
5.	Other Nonfunctional Requirements	18
o	5.1 Performance Requirements	18
o	5.2 Safety Requirements	18
o	5.3 Security Requirements	18
o	5.4 Software Quality Attributes	18
o	5.5 Business Rules	19
6.	Other Requirements	19
	Appendix A: Glossary	19
	Appendix B: Analysis Models	20
	Appendix C: To Be Determined List	58

Revision History

Name	Date	Reason for Changes	Version
Brison B	11-06-2026	Initial SRS Creation	1.0

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for the Smart Academic Feedback & Improvement Management System.

This system allows students to provide feedback, faculty to manage questions and suggestions, and HOD/Admin to monitor academic performance and improvement activities in an institution.

1.2 Document Conventions

- SRS refers to Software Requirements Specification.
- Admin refers to System Administrator.
- FR refers to Functional Requirement.
- MFR refers to Non-Functional Requirement.
- High Priority requirements are mandatory.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Project Developers
- System Testers
- Project Managers
- Faculty Members
- College Administrators

Readers should begin with the Introduction and Overall Description sections before proceeding to System Features and Requirements.

1.4 Product Scope

The AFIMS system is a web-based application designed for educational institutions to manage academic feedback dynamically.

It allows:

- Students to submit feedback on faculty and courses
- Faculty to create, edit, and delete feedback questions
- HOD to approve questions and generate reports
- Admin to manage users and system configuration

The system improves academic quality through continuous feedback and structured improvement tracking.

1.5 References

1. IEEE Recommended Practice for Software Requirements Specifications.
2. Software Engineering by Roger S. Pressman.
3. UML Modeling Standards.

2. Overall Description

2.1 Product Perspective

The system is a standalone web-based application designed for educational institutions. It provides online feedback collection and report generation.

2.2 Product Functions

- - User Authentication (Student, Faculty, HOD, Admin)
- - Dynamic Question Management
- - Student Feedback Submission
- - Faculty Improvement Suggestions
- - HOD Approval System
- - Admin User Management
- - Report Generation and Analytics

2.3 User Classes and Characteristics

Student Use Case Description

UC-01: Submit Feedback

- **Primary Actor:** Student
- **Secondary Actor:** Database Server
- **Trigger:** Student clicks an active evaluation link on their dashboard.
- **Preconditions:** Student is logged in: an uncompleted evaluation survey exists.

- **Postconditions:** Feedback is saved anonymously; survey is marked complete for that student.
 - **Main Success Scenario:**
 1. Student requests pending evaluations.
 2. System displays active evaluation forms.
 3. Student selects a form and fills out ratings and comments.
 4. Student clicks "Submit Feedback".
 5. System removes personal identifying info from the payload.
 6. System saves anonymized entries to the database.
 7. System flags the survey as "Completed" for this student.
 8. System displays a submission confirmation message.
 - **Exceptions:**
 1. *3a. Survey Expired:* System blocks write access and displays an error message.
 2. *5a. Session Timeout:* System invalidates input and redirects to login screen.
-

UC-02: View Survey and Questions

- **Primary Actor:** Student
 - **Secondary Actor:** None
 - **Trigger:** Student accesses the evaluation directory.
 - **Preconditions:** Student is logged in.
 - **Postconditions:** Survey instructions and questions are displayed.
 - **Main Success Scenario:**
 1. Student navigates to the evaluation tab.
 2. System queries active surveys matching student's enrolled courses.
 3. System displays survey list with deadlines and question counts.
 4. Student clicks a survey to preview content.
 5. System renders instructions and the question structure.
 - **Exceptions:**
 1. *2a. No Surveys:* System displays "No Pending Course Evaluations Found".
-

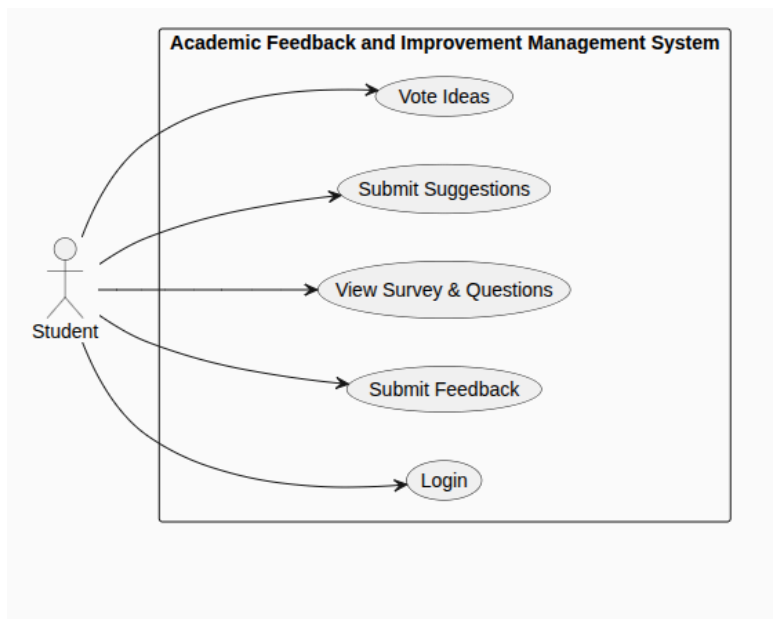
UC-03: Submit Suggestions

- **Primary Actor:** Student
- **Secondary Actor:** Notification Service
- **Trigger:** Student clicks "New Suggestion" on the public board.
- **Preconditions:** Student is logged in.
- **Postconditions:** Suggestion is published to the public board for peer review.
- **Main Success Scenario:**
 1. Student opens the suggestions dashboard.
 2. Student selects a category (e.g., Campus, Curriculum, Facilities).
 3. Student inputs a title and description.
 4. Student clicks "Publish".
 5. System runs content filters on text fields.
 6. System publishes the suggestion to the public board.
 7. System triggers a notification alert to peers.
- **Exceptions:**

1. *3a. Blank Fields:* System highlights required text boxes and blocks submission.
 2. *5a. Policy Violation:* System flags profanity/XSS, halting submission.
-

UC-04: Vote Ideas

- **Primary Actor:** Student
 - **Secondary Actor:** None
 - **Trigger:** Student clicks upvote or downvote next to a peer's suggestion.
 - **Preconditions:** Student is logged in; suggestion is open for review.
 - **Postconditions:** Global vote count updates instantly; UI displays new score.
 - **Main Success Scenario:**
 1. Student views ideas on the suggestion board.
 2. Student clicks an upvote or downvote icon on an item.
 3. System verifies student hasn't voted on this item before.
 4. System registers the user-item link to prevent duplication.
 5. System updates the global vote counter (+1 or -1).
 6. System refreshes the display widget instantly.
 - **Exceptions:**
 1. *4a. Duplicate Vote:* System retracts the user's previous vote and updates count.
-



Faculty Use Case Descriptions

UC-05: Manage questions

- **Primary Actor:** Faculty Member
 - **Secondary Actor:** HOD Workflow Router
 - **Trigger:** Faculty navigates to their question bank section.
 - **Preconditions:** Faculty is logged in.
 - **Postconditions:** Questions are updated with a "Pending Approval" status.
 - **Main Success Scenario:**
 1. Faculty requests to add, edit, or delete a question.
 2. System opens the item editing workspace.
 3. Faculty performs modifications (e.g., updates text or types).
 4. Faculty clicks "Submit for Approval".
 5. System saves changes with status set to "Pending Approval".
 6. System alerts the HOD workflow queue of a new request.
 - **Exceptions:**
 1. *3a. Missing Choices:* System rejects closed-ended items submitted without options.
-

UC-06: View Feedback

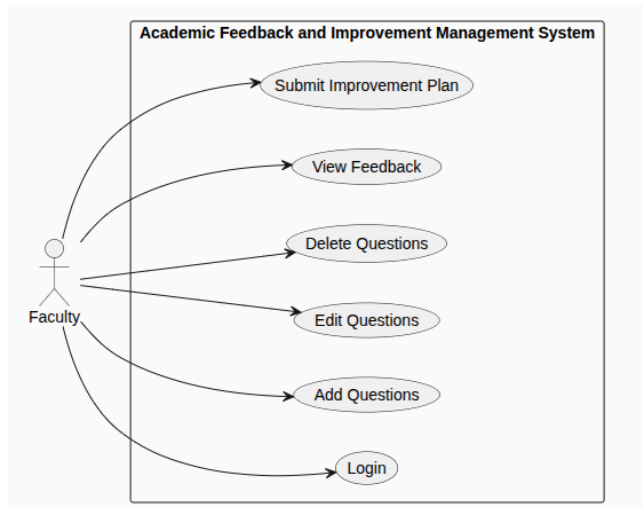
- **Primary Actor:** Faculty Member
 - **Secondary Actor:** Analytics Engine
 - **Trigger:** Faculty clicks the evaluation results tab.
 - **Preconditions:** Faculty is logged in; campaign has officially ended.
 - **Postconditions:** Aggregated data charts and comments are displayed.
 - **Main Success Scenario:**
 1. Faculty selects a finished course evaluation campaign.
 2. System verifies that the campaign timeline is closed.
 3. System pulls anonymized numeric ratings and text comments.
 4. System generates statistical charts (e.g., bar graphs).
 5. System displays the compiled analytics report on screen.
 - **Exceptions:**
 1. *2a. Campaign Active:* System blocks view access until the campaign window closes.
-

UC-07: Submit Improvement Plan

- **Primary Actor:** Faculty Member
- **Secondary Actor:** HOD Review Queue
- **Trigger:** Faculty clicks "Create Action Plan" on an evaluation report.
- **Preconditions:** Faculty is logged in; evaluation feedback has been reviewed.
- **Postconditions:** Plan is routed directly to the HOD dashboard queue.
- **Main Success Scenario:**
 1. Faculty opens the Improvement Plan template.
 2. Faculty drafts action items and strategies to fix low-scoring areas.
 3. Faculty defines implementation timelines and milestone goals.
 4. Faculty clicks "Route to HOD".
 5. System saves the file and updates its state to "Submitted".
 6. System populates the file inside the HOD's review queue.

- **Exceptions:**

1. *1a. Feedback Unread:* System restricts plan creation until the analytics report is opened.



HOD Use Case Descriptions

UC-08: Create Survey Campaign

- **Primary Actor:** Head of Department (HOD)
- **Secondary Actor:** Messaging Server
- **Trigger:** HOD clicks "Launch New Evaluation Schedule".
- **Preconditions:** HOD is logged in.
- **Postconditions:** Survey schedule becomes active for targeted student rosters.
- **Main Success Scenario:**
 1. HOD selects target departments, semesters, and course lists.
 2. HOD sets campaign start dates, end deadlines, and reminder intervals.
 3. HOD links approved departmental evaluation questions.
 4. HOD clicks "Deploy Evaluation Campaign".
 5. System saves and marks the survey campaign status as active.
 6. System pushes notification templates out to notify eligible student dashboards.
- **Exceptions:**
 1. *2a. Date Conflict:* System rejects deadlines set prior to or same as start dates.

UC-09: Approve Questions

- **Primary Actor:** Head of Department (HOD)
- **Secondary Actor:** Notification Service

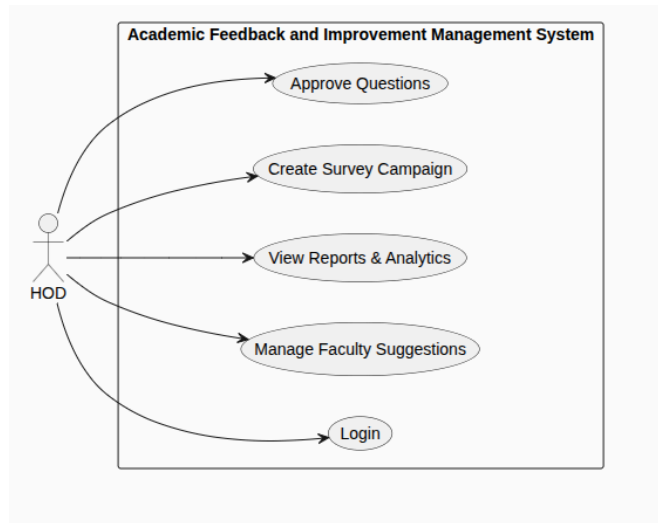
- **Trigger:** HOD opens an item from their pending approvals index.
 - **Preconditions:** HOD is logged in: faculty has submitted questions.
 - **Postconditions:** Question status transitions to "Live" or "Rejected".
 - **Main Success Scenario:**
 1. HOD opens incoming custom question edits from faculty.
 2. HOD reviews content and clicks the "Approve" controller.
 3. System updates the status tag of the question to "Live".
 4. System saves changes, appending the item to the main active pool.
 5. System dispatches a status notification to the originating instructor portal.
 - **Exceptions:**
 1. *3a. Missing Comments on Rejection:* System demands comment logs if "Reject" is clicked.
-

UC-10: Manage Faculty Suggestions

- **Primary Actor:** Head of Department (HOD)
 - **Secondary Actor:** None
 - **Trigger:** HOD opens an improvement plan folder in their workflow view.
 - **Preconditions:** HOD is logged in: faculty has forwarded action plans.
 - **Postconditions:** Improvement plan status changes to "Reviewed".
 - **Main Success Scenario:**
 1. HOD opens a faculty member's submitted plan.
 2. HOD reviews proposed interventions and timelines.
 3. HOD writes performance remarks or changes into the notes field.
 4. HOD clicks "Mark as Reviewed".
 5. System updates plan lifecycle state configuration to "Reviewed".
 6. System shares HOD feedback remarks back with the instructor's workspace.
 - **Exceptions:**
 1. *5a. Network Dropout:* System warns HOD that changes were not saved due to loss of connection.
-

UC-11: View Reports & Analytics

- **Primary Actor:** Head of Department (HOD)
 - **Secondary Actor:** Analytics Engine
 - **Trigger:** HOD selects the departmental insights tab.
 - **Preconditions:** HOD is logged in.
 - **Postconditions:** Trend data and department-wide charts are displayed.
 - **Main Success Scenario:**
 1. HOD requests evaluation performance summaries for the term.
 2. System tells the analytics engine to index evaluation data rows.
 3. System calculates averages, scores, and track trends.
 4. System builds overview charts and faculty comparison rankings.
 5. System renders the interactive reports to the HOD's screen.
 - **Exceptions:**
 1. *2a. Low Data Density:* System hides graphs if sample numbers threaten anonymity.
-



Admin Use Case Descriptions

UC-12: Manage Users

- **Primary Actor:** Administrator
- **Secondary Actor:** IAM Identity Store
- **Trigger:** Admin opens the account provisioning panel.
- **Preconditions:** Admin is logged in.
- **Postconditions:** User accounts are successfully created, updated, or deleted.
- **Main Success Scenario:**
 1. Admin clicks on an action: Create, Update, or Delete user profiles.
 2. Admin enters account parameters (Name, Email, Role permissions).
 3. Admin clicks "Commit Changes".
 4. System modifies the identity data configurations accordingly.
 5. System synchronizes updates across the IAM identity database.
 6. System appends transaction logs to the security archive tracking trail.
- **Exceptions:**
 1. *1a. Self-Deletion:* System blocks requests trying to delete the active admin account.

UC-13: System Configuration

- **Primary Actor:** Administrator
- **Secondary Actor:** Environment Infrastructure
- **Trigger:** Admin alters parameters inside platform setup consoles.
- **Preconditions:** Admin is logged in.
- **Postconditions:** System configuration variables apply globally.
- **Main Success Scenario:**
 1. Admin alters properties (e.g., session timeout limits, SMTP server rules).
 2. Admin clicks "Apply Configuration Changes".
 3. System checks values for correct syntax structure rules.
 4. System rewrites configuration files on the registry.
 5. System refreshes configuration states across all portal nodes smoothly.

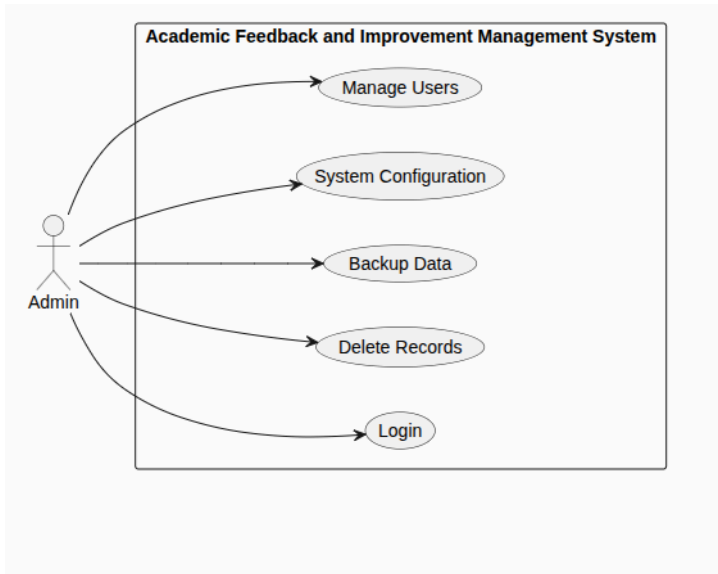
- **Exceptions:**
 - 1. *3a. Connection Check Failure:* System rolls back edits if new server strings fail validation check paths.
-

UC-14: Backup Data

- **Primary Actor:** Administrator
 - **Secondary Actor:** Cloud Storage Engine
 - **Trigger:** Admin executes a manual backup or automated cron-task window triggers.
 - **Preconditions:** Admin is logged in.
 - **Postconditions:** A secure snapshot file is stored in separate cloud folders.
 - **Main Success Scenario:**
 1. Admin starts the database backup command link.
 2. System locks active sessions into read-only mode briefly.
 3. System builds a compressed, encrypted copy snapshot file of the database.
 4. System transfers the file backup to cloud storage servers safely.
 5. System unlocks normal table reads and writes.
 6. System logs success details (file hash, sizes, completion timestamps).
 - **Exceptions:**
 - 1. *4a. Storage Full:* System halts deployment, rolls tables back, and fires failure alerts.
-

UC-15: Delete Records

- **Primary Actor:** Administrator
 - **Secondary Actor:** Database Server
 - **Trigger:** Admin opens data retention cleanup features.
 - **Preconditions:** Admin is logged in.
 - **Postconditions:** Outdated system log rows are permanently dropped.
 - **Main Success Scenario:**
 1. Admin filters specific historical log sets to drop.
 2. Admin re-enters credential keys to confirm critical deletion steps.
 3. Admin runs the permanent deletion tool.
 4. System drops chosen lines from underlying data rows.
 5. System rebuilds database indexes to free drive space blocks.
 6. System inserts a permanent change notice in security audit track files.
 - **Exceptions:**
 - 1. *4a. Constraint Conflict:* Database blocks cleanup rules if lines are actively referenced by current active evaluations.
-



2.4 Operating Environment

- - Operating System: *Windows 10/11*
- - Backend: *Python Flask*
- - Frontend: *HTML, CSS, Bootstrap*

- - Database: *MySQL (future implementation)*
- - Browser: *Chrome, Edge, Firefox*

2.5 Design and Implementation Constraints

- Internet connection required.
- *MySQL* database must be available.
- User authentication mandatory.

2.6 User Documentation

- User *Manual*
- Installation Guide
- Administrator Guide

2.7 Assumptions and Dependencies

- Students possess valid login credentials.
- Faculty data is maintained by Admin.
- Internet connectivity is available.

3. External Interface Requirements

This section details the specific requirements for how the **AFIMS** application interacts with human users, hardware devices, database storage software, and internal networking communication protocols.

3.1 User Interfaces

The user interface layout follows a responsive, mobile-first design philosophy using modern frontend frameworks like *HTML5*, *CSS3*, and *JavaScript* components.

- **General GUI Layout Constraints:**
- **Persistent Navigation Drawer:** All authorized screens must feature a top/side navbar displaying the current active context,

system logo, notification bell, and a logout option.

- **Role-Based Views:** Dynamic dashboard widgets customize layouts on success login. Students see active survey notifications: Faculty view active draft forms: HODs view core analytic trend visualizations.
- **Input & Validation Rules:** Text field fields validate formatting sequentially. Forms throw immediate inline error notifications (red bold text tags) for invalid email strings or blank entries.
- **Screen Ergonomics:** Background canvas must support white/dark mode toggle variables with color-blind friendly contrasts matching global web accessibility benchmarks.

3.2 Hardware Interfaces

The application manages interface boundaries on standard web infrastructure without demanding hyper-specialized peripheral setups.

- **Client Devices:** Fully supports standard workstation terminals, tablets, and smartphone variants with minimum hardware resolutions of 320x568 pixels up to 4K ultra-wide configurations.
- **Server Deployment Base:** Runs dynamically on host machines provisioned with a minimum processing capacity of 2 Core vCPU setups, 4GB System RAM, and 40GB solid-state drive (SSD) arrays to accommodate heavy concurrent feedback logging periods.
- **Network Components:** Requires persistent interaction with local WiFi gateway systems, Ethernet switches, and firewall hardware arrays to ensure secure traffic delivery paths.

3.3 Software Interfaces

AFIMS maps data dependencies explicitly across external execution runtime environments and enterprise database libraries.

Component Name	Technical Version	Interface Definition / Purpose
Operating System	Linux Ubuntu Server 22.04 LTS / Windows 11	Underlying core environment hosting execution containers.
Runtime Engine	Node.js v18.x / Java JDK 17	Core script environment running the backend business logic.
Database Server	MySQL Community Server 8.0	Manages storage instances for schemas mapped inside the class diagram (Student, SurveyCampaign, FeedbackResponse).
Authentication Module	JWT (JSON Web Tokens) Standard	Generates cryptographically secure base64 string access tokens to track user sessions.

3.4 Communication Interfaces

Secure connection protocols ensure internal endpoints remain decoupled while transmitting operations safely over active sockets.

- **Hypertext Transfer Protocol (HTTPS):** All browser requests use TLS 1.3 encryption layers over standard TCP port 443 to block unauthorized payload sniffers.
- **Asynchronous API Contracts:** Systems leverage the RESTful JSON communication architecture to manage structured object schemas (e.g., streaming `Suggestion` votes or saving a new `ImprovementPlan` entry).
- **Database Driver Connectivity:** Internal web application instances map to local storage infrastructure using highly performant, connection-pooled JDBC/ODBC connection strings.
- **Mail Server Gateway:** Integrates directly with SMTP mail protocols (Simple Mail Transfer Protocol) using TLS over port 587

to dispatch automatic reminder logs to students for incomplete `SurveyCampaign` dates.

4. System Features

4.1 Student Login

4.1.1 Description and Priority

This feature allows students to authenticate themselves securely to access the system and view their personalized dashboard. [i]

- **Priority:** High

4.1.2 Stimulus/Response Sequences

- **Stimulus:** Student enters their institutional roll number and password on the login page and clicks "Submit".
- **Response:** System validates credentials against the database, creates a secure session, and redirects the student to the feedback dashboard.
- **Stimulus:** Student enters incorrect credentials.
- **Response:** System displays an error message ("Invalid Credentials") and prompts the user to try again. [i]

4.1.3 Functional Requirements

- **FR-1.1:** The system must authenticate student login requests using institutional credentials.
- **FR-1.2:** The system must lock a student account temporarily after five consecutive failed login attempts.
- **FR-1.3:** The system must provide a "Forgot Password" self-service link using verified institutional email IDs. [i]

4.2 Feedback Submission

4.2.2 Description and Priority

This feature enables students to fill out and submit feedback forms for their respective courses and faculty members anonymously.

- **Priority:** High

4.2.2 Stimulus/Response Sequences

- **Stimulus:** Student selects an active course/faculty combination from their dashboard.

- **Response:** System displays the approved questionnaire corresponding to that specific course.
- **Stimulus:** Student fills all mandatory fields and clicks "Submit Feedback".
- **Response:** System validates completeness, saves responses into an anonymized database table, and marks the course feedback as "Completed" for that student.

4.2.3 Functional Requirements

- **FR-2.1:** The system must hide the identity of the student submitting the feedback to ensure absolute anonymity.
- **FR-2.2:** The system must prevent students from submitting feedback more than once per course per semester.
- **FR-2.3:** The system must allow students to save responses as a draft before final submission. [1]

4.3 Report Generation

4.3.1 Description and Priority

This feature allows Heads of Departments (HODs) and Administrators to generate analytical visual graphs and summaries based on collected feedback scores.

- **Priority:** High

4.3.2 Stimulus/Response Sequences

- **Stimulus:** HOD selects a faculty member, semester, and course, then clicks "Generate Report".
- **Response:** System aggregates feedback data, calculates cumulative performance indices, and renders visual charts.
- **Stimulus:** Admin selects "Export Report".
- **Response:** System compiles the visual and textual data into a downloadable PDF/Excel file format.

4.3.3 Functional Requirements

- **FR-3.1:** The system must compute weighted average metrics for quantitative feedback parameters automatically.
- **FR-3.2:** The system must restrict report access strictly to authorized HODs, Principals, and Admins.
- **FR-3.3:** The system must export reports cleanly into standard `.pdf` and `.xlsx` formats.

4.4 Faculty Management

4.4.1 Description and Priority

This feature allows faculty members to interact with the feedback lifecycle by managing potential survey questions and viewing their final authorized scores.

- **Priority:** Medium

4.4.2 Stimulus/Response Sequences

- **Stimulus:** Faculty logs in and adds a new feedback question for their subject.
- **Response:** System saves the question with a status flag set to "Pending HOD Approval".
- **Stimulus:** HOD reviews the dashboard and clicks "Approve Question".
- **Response:** System changes the status flag to "Active" and includes it in the active student survey.

4.4.3 Functional Requirements

- **FR-4.1:** The system must allow faculty members to draft, edit, and delete self-proposed feedback questions.
- **FR-4.2:** The system must enforce a review workflow requiring HOD sign-off before any question becomes live to students.
- **FR-4.3:** The system must provide faculty with a private dashboard to view approved performance feedback scores.

4.5 Student Management

4.5.1 Description and Priority

This module allows the System Administrator to manage student registration records, academic enrollments, and course allocations. [i]

- **Priority:** Medium

4.5.2 Stimulus/Response Sequences

- **Stimulus:** Admin uploads a `.csv` student roster file and clicks "Import".
- **Response:** System parses the file, creates user profiles, links students to correct batches, and outputs a success summary log.

4.5.3 Functional Requirements

- **FR-5.1:** The system must support bulk student account creation using standard comma-separated value format spreadsheets.
 - **FR-5.2:** The system must allow Admins to map specific student batches to their relevant semester course codes.
 - **FR-5.3:** The system must track active/inactive statuses of student accounts for security and auditing purposes.
-

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Login response time shall be less than 3 seconds.
- Reports shall be generated within 5 seconds.

5.2 Safety Requirements

- Prevent accidental deletion of feedback records.
- Backup facility must be provided.

5.3 Security Requirements

- Password authentication required.
- Unauthorized access shall be restricted.
- Data shall be securely stored.

5.4 Software Quality Attributes

Reliability

System availability should be 99%.

Maintainability

System should support future modifications.

Usability

User-friendly graphical interface.

Portability

System should run on different operating systems.

5.5 Business Rules

- One student can submit feedback only once per faculty per semester.
- Only Admin can generate final reports.
- Faculty cannot modify submitted feedback.

6. Other Requirements

Database Requirements

Tables:

- Student
- Faculty
- Feedback
- Department
- Admin

Primary and Foreign Keys shall maintain data integrity.

Appendix A: Glossary

Term	Description
SRS	Software Requirements Specification
Admin	Administrator
DBMS	Database Management System
FR	Functional Requirement

Appendix B: Analysis Models

B.1.Class Diagram

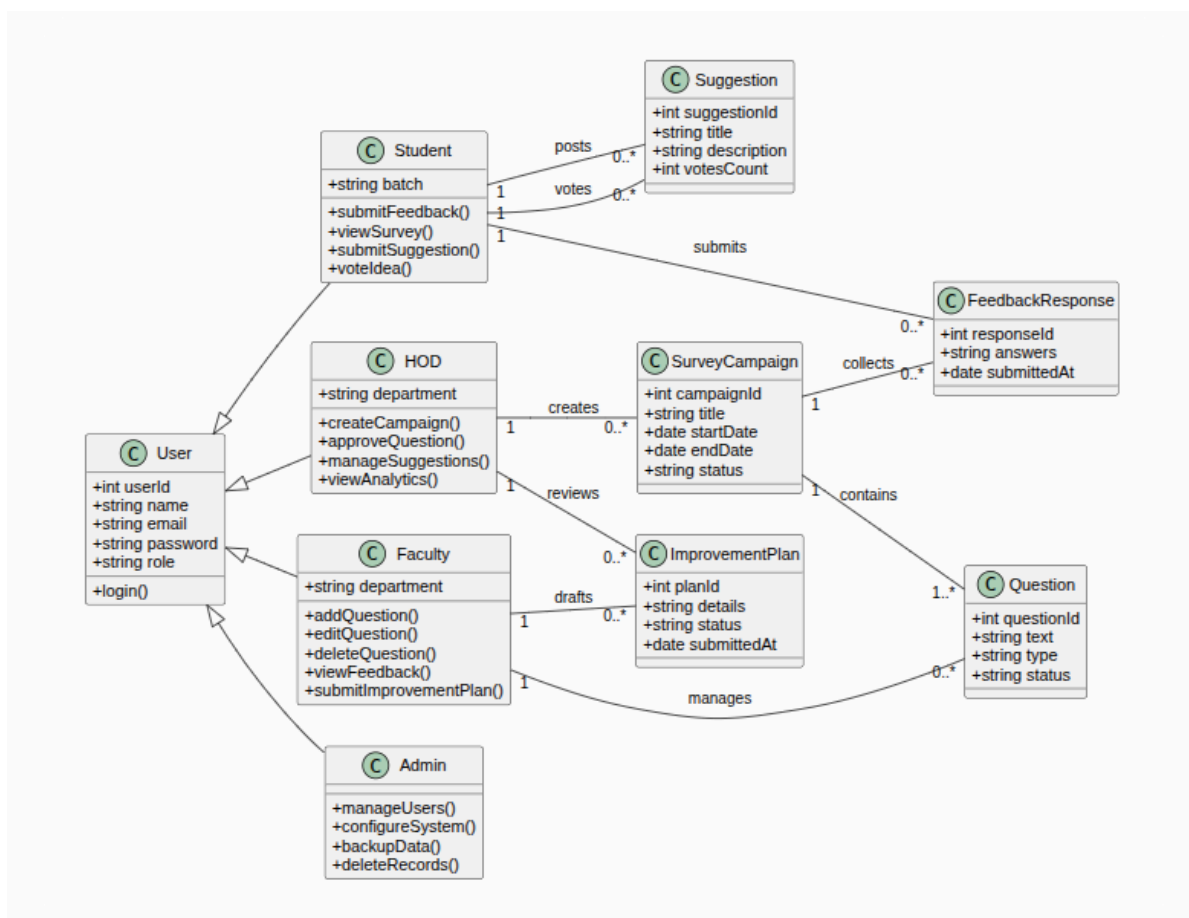


Figure B.1: System Class Diagram Model

B.1.1 Overview of Domain Model

The structural model depicted in Figure B.1 represents the logical schema of the Feedback and Suggestion Management System. The architecture employs an Object-Oriented paradigm using a centralized inheritance structure to segregate user privileges while maintaining common authentication data. The relationships enforce data integrity across feedback collections, survey management, and subsequent action plans.

B.1.2 Class Dictionary and Data Definitions

The following specifications define the functional responsibilities, core attributes, and behaviors for each entity within the domain model.

Class Name	Responsibility	Key Attributes	Core Methods / Operations
User	Serves as the abstract base class for system authentication and general profile management.	userId, name, email, password, role	login()
Student	Inherits from User. Represents the primary end-user who provides feedback and community suggestions.	batch	submitFeedback(), viewSurvey(), submitSuggestion(), voteIdea()
Faculty	Inherits from User. Represents academic staff responsible for reviewing feedback and generating solutions.	department	addQuestion(), editQuestion(), viewFeedback(), submitImprovementPlan()
HOD	Inherits from User. Represents departmental heads who oversee academic metrics, campaigns, and plans.	department	createCampaign(), approveQuestion(), manageSuggestions(), viewAnalytics()
Admin	Inherits from User. Handles core system maintenance, backups, and user access control lists.	None (Inherited)	manageUsers(), configureSystem(), backupData(), deleteRecords()
SurveyCampaign	Manages the life cycle of a specific feedback collection event.	campaignId, title, startDate, endDate, status	Data Entity
Question	Defines individual evaluation metrics attached to a survey campaign.	questionId, text, type, status	Data Entity
FeedbackResponse	Encapsulates a student's completed responses to a specific survey campaign.	responseId, answers, submittedAt	Data Entity
Suggestion	Stores community-driven ideas or grievances posted by students for peer voting.	suggestionId, title, description, votesCount	Data Entity
ImprovementPlan	Records formal action points drafted by faculty and reviewed by HODs to resolve feedback issues.	planId, details, status, submittedAt	Data Entity

B.2 Activity Diagrams

The behavioral models in this section depict the operational workflows of the Academic Feedback and Improvement Management System (AFIMS). These activity diagrams map the sequential execution of business logic, conditional decision branches, and concurrent processes across the specialized user roles defined in the domain schema (Section B.1).

B.2.1 Student Activity Diagram (Feedback Submission & Suggestion Lifecycle)

A. Visual Representation

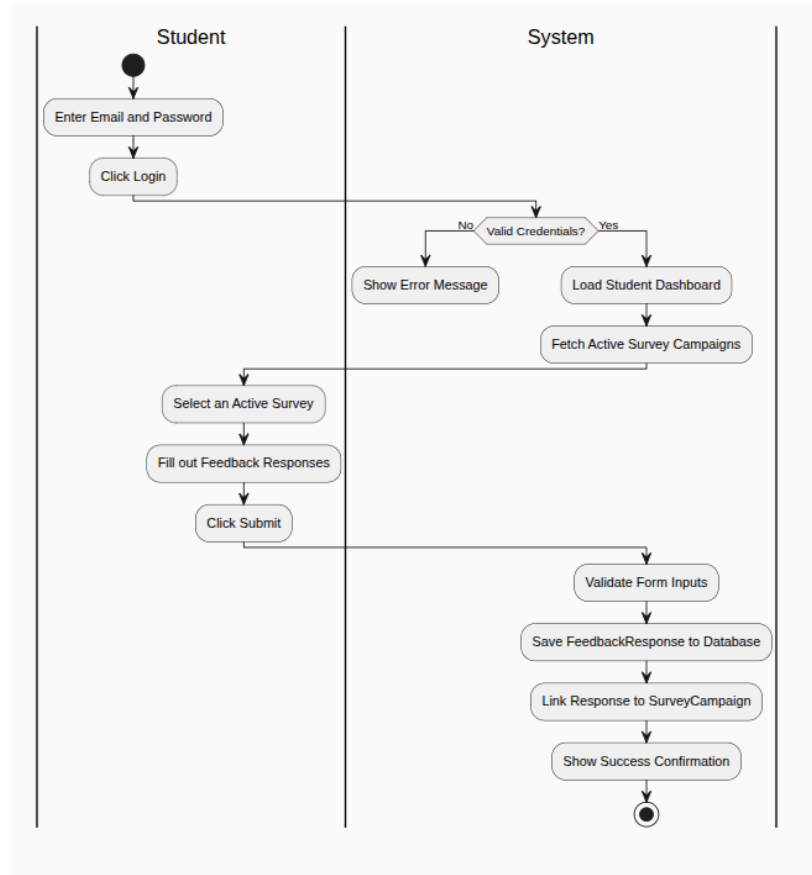


Figure B.2.1: Student Activity Diagram Flow

B. Behavioral Specifications

- **Pre-conditions:** The `Student` is securely authenticated: at least one `SurveyCampaign` status is set to "Active".
- **Post-conditions:** `FeedbackResponse` is committed to the database: new `Suggestion` record is initialised with `votesCount` equal to zero.

C. Execution Sequence

1. **Authentication:** Student logs into AFIMS via the `User` base authentication method.
2. **Navigation Choice:** The system branches into two major concurrent options based on user selection:
 - **Path A (Survey Participation):** Student selects `viewSurvey()`, fetches active evaluations, populates numeric/text answers, and executes `submitFeedback()`.
 - **Path B (Suggestion Forum):** Student invokes `submitSuggestion()` by entering a text title and description for an academic grievance or community idea.
3. **Peer Interaction:** For existing suggestions, the student can browse the community board and execute `voteIdea()`.

incrementing the `votesCount` attribute.

4. **Final State:** The system securely hashes response tokens to maintain anonymity, logs data entities, and gracefully terminates the user session.

B.2.2 Teacher Activity Diagram (Feedback Review & Improvement Planning)

A. Visual Representation

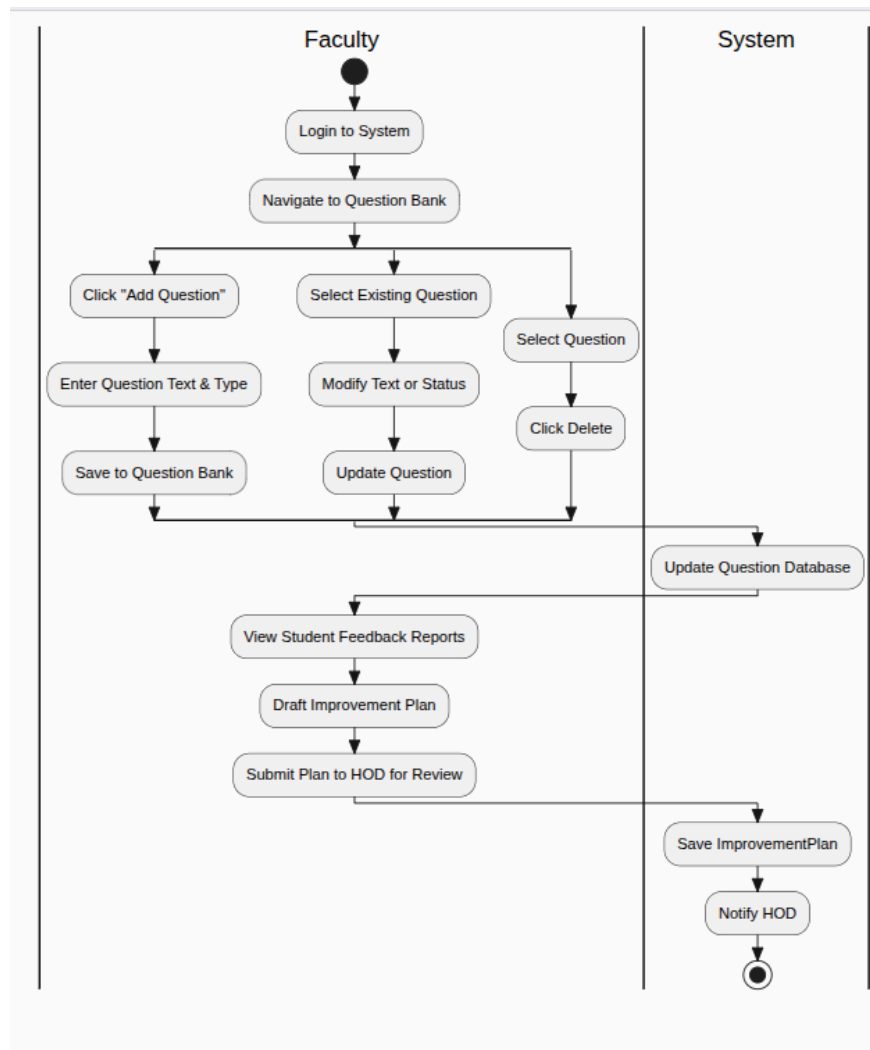


Figure B.2.2: Teacher Activity Diagram Flow

B. Behavioral Specifications

- **Pre-conditions:** The `Faculty` member is authenticated: a concluded `SurveyCampaign` contains generated data responses linked to their assigned courses.
- **Post-conditions:** An `ImprovementPlan` entity is instantiated, flagged as "Pending Review", and assigned to the departmental queue.

C. Execution Sequence

1. **Initiation:** Faculty logs into the system dashboard and invokes `viewFeedback()`.
2. **Data Retrieval:** System queries `FeedbackResponse` records, compiles anonymized quantitative metrics, and displays performance indicators.
3. **Analysis Phase:** Faculty evaluates structural problem areas (e.g., resource availability, syllabus pacing issues).
4. **Plan Draft Generation:** Faculty executes `submitImprovementPlan()`, populating specific `details` (action points, milestones, and target timelines).
5. **Submission Routing:** The system updates the plan status to "Submitted to HOD" and routes the workflow to the departmental head's oversight interface.

B.2.3 HOD Activity Diagram (Campaign Control & Quality Assurance Oversight)

A. Visual Representation

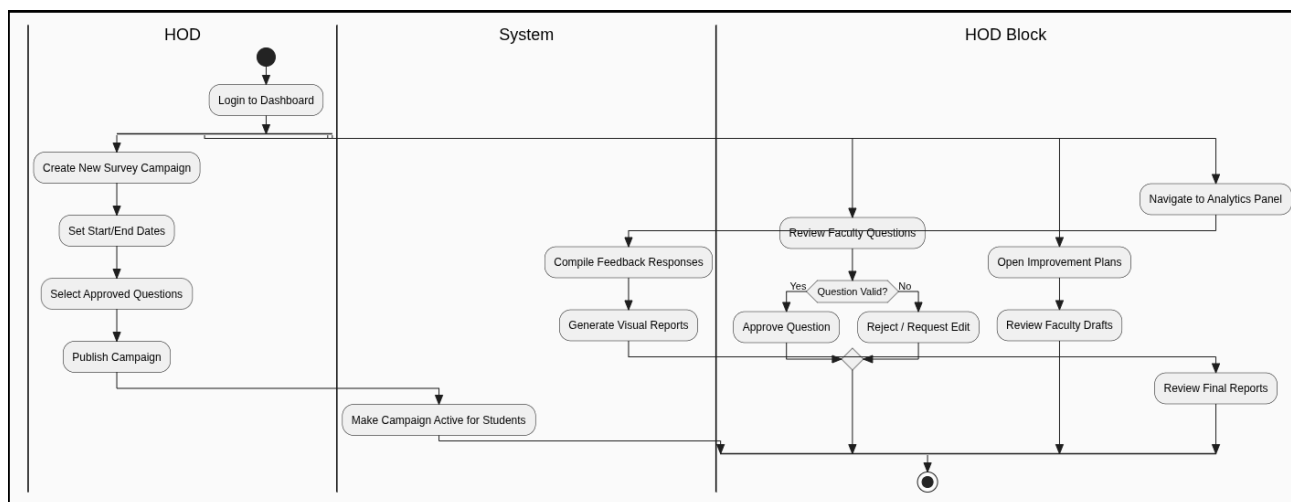


Figure B.2.3: HOD Activity Diagram Flow

B. Behavioral Specifications

- **Pre-conditions:** `HOD` user is authenticated: system contains pending configuration metrics, metrics data from faculty, or suggestions from students.
- **Post-conditions:** `SurveyCampaign` transitions to an active state, community suggestions are moderated, and faculty improvement plans are approved or rejected.

C. Execution Sequence

1. **Campaign Management:** HOD initializes `createCampaign()`, defining structural parameters like `startDate`, `endDate`, and campaign targets.
2. **Question Validation:** HOD invokes `approveQuestion()` to inspect evaluation metrics drafted by staff.
 - **Decision Branch (Validation Passed?):** If yes, question status sets to "Approved"; if no, it is returned to Faculty with an "Edit Flag".
3. **Community Moderation:** HOD monitors high-voting items via `manageSuggestions()`, escalating urgent feedback points directly

into operational reviews.

4. **Plan Authorization:** HOD reviews pending `ImprovementPlan` items using `viewAnalytics()`.
- **Approval Branch:** If details are valid, plan status shifts to "Approved" (triggering implementation tracking). If invalid, it reverts to Faculty for revisions.

B.2.4 Admin Activity Diagram (System Maintenance & Access Control)

A. Visual Representation

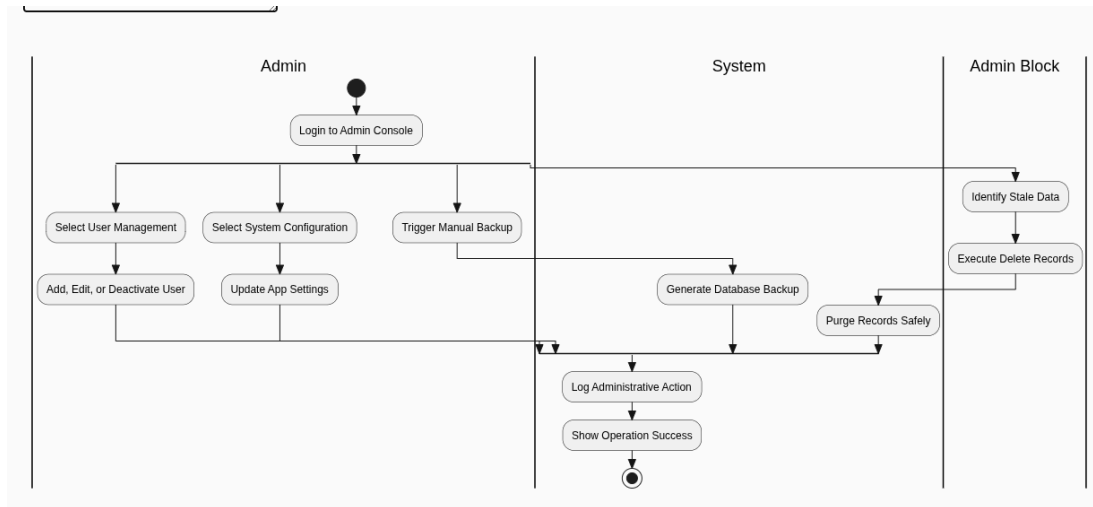


Figure B.2.4: Admin Activity Diagram Flow

B. Behavioral Specifications

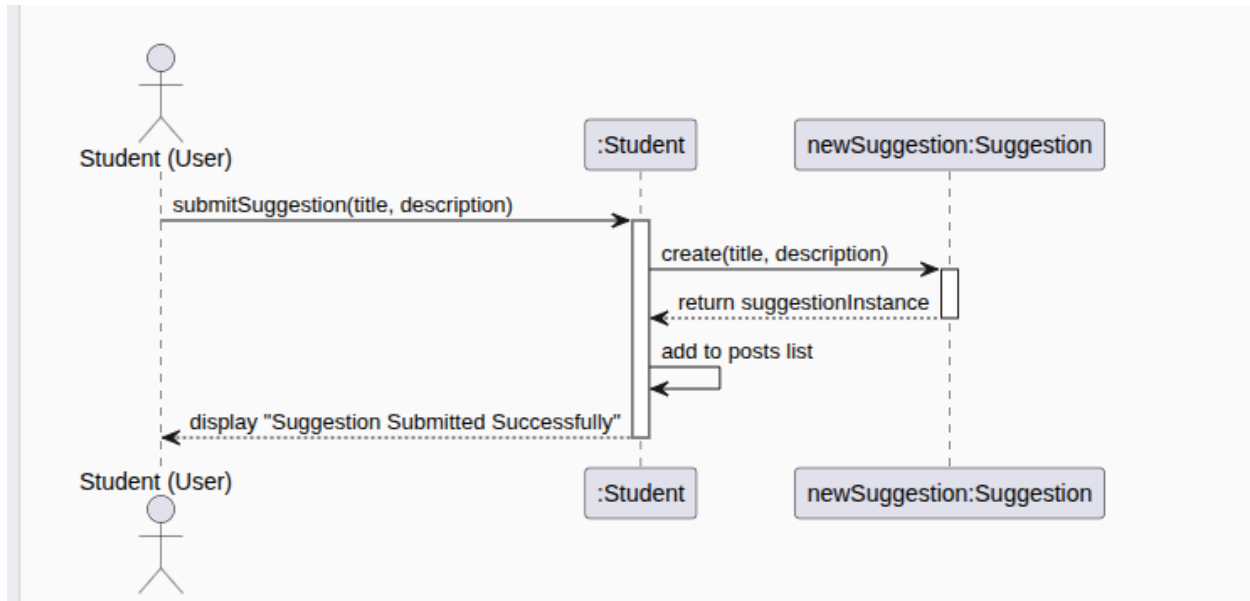
- **Pre-conditions:** `Admin` user provides root-level authentication credentials; access to physical/cloud data stores is established.
- **Post-conditions:** System logs are synchronized, user profiles match current institutional directories, and an encrypted system backup point is recorded.

C. Execution Sequence

1. **System Entry:** Admin bypasses multi-tenant layer to access the master configuration panel.
2. **Operational Routing:** Admin selects from three core asynchronous maintenance options:
 - **User Management:** Invokes `manageUsers()` to onboard new batches, modify privilege levels, or deactivate invalid user IDs.
 - **System Configuration:** Runs `configureSystem()` to change global survey operational policies or adjust voting thresholds.
 - **Database Lifecycle Management:** Triggers `backupData()` to compress and isolate historical survey indices, or processes `deleteRecords()` to purge expired system sessions.
3. **Verification Loop:** System runs automated check routines to guarantee that underlying relational tables and inheritance classes remain uncorrupted.
4. **Final State:** The system updates master audit trail logs with Admin actions and seals data access points.

B.3.Sequence Diagram

Figure X.1: Suggestion Submission Sequence Diagram



1. Purpose

This sequence diagram illustrates the step-by-step behavioral interaction when a user submits a new suggestion within the system.

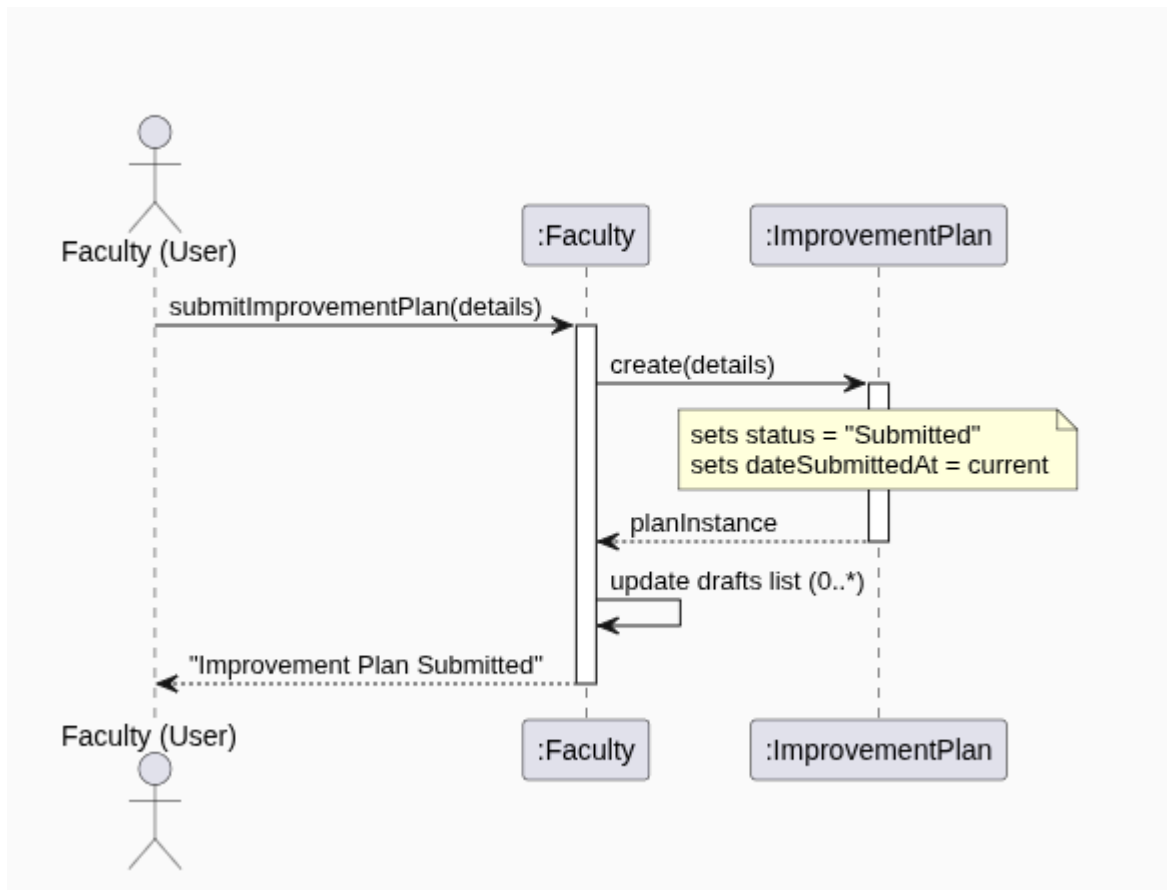
2. Participants and Objects

- **Student (User):** The external actor initiating the submission process.
- **:Student:** The controller or boundary object managing student-specific actions.
- **newSuggestion:Suggestion:** The entity object created to store the submission data.

3. Execution Flow Description

1. **Trigger:** The `Student (User)` invokes `submitSuggestion(title, description)`.
2. **Instantiation:** The `:Student` object calls `create(title, description)` to instantiate `newSuggestion`.
3. **Confirmation:** The `newSuggestion` object returns the `suggestionInstance` to the controller.
4. **Internal Update:** The `:Student` object executes an internal call to `add to posts list`.
5. **UI Response:** The system displays the confirmation message `"Suggestion Submitted Successfully"` back to the user.

Figure X.2: Improvement Plan Submission Sequence Diagram



1. Purpose

This sequence diagram details the behavioral interaction and state changes that occur when a faculty member submits an improvement plan.

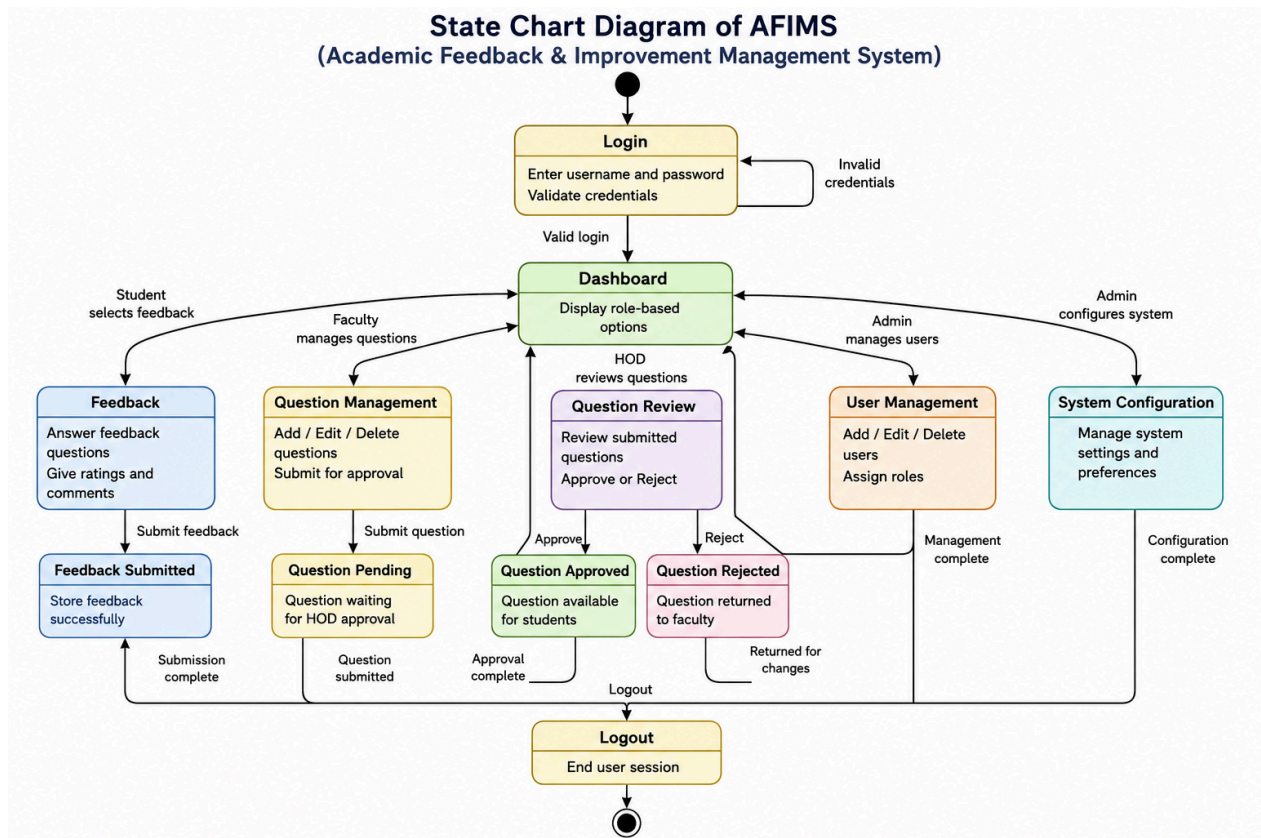
2. Participants and Objects

- **Faculty (User):** The external academic actor initiating the improvement plan submission.
- **:Faculty:** The boundary/controller object managing operations on behalf of the faculty user.
- **:ImprovementPlan:** The entity object generated to store the plan's contents and trace its lifecycle.

3. Execution Flow Description

- 1. Trigger:** The `Faculty (User)` triggers the action by sending `submitImprovementPlan(details)` to the controller.
- 2. Instantiation & State Initialization:** The `:Faculty` controller invokes `create(details)`, initializing a new `:ImprovementPlan` instance. During creation, the plan internally updates its attributes to `status = "Submitted"` and `dateSubmittedAt = current`.
- 3. Confirmation:** The newly created `:ImprovementPlan` returns its reference (`planInstance`) back to the controller.
- 4. Internal Update:** The `:Faculty` controller runs a self-call to `update drafts list (0..*)` to synchronize the list elements.
- 5. UI Response:** The system returns a textual confirmation `"Improvement Plan Submitted"` to the user's interface.

B.4.State Chart Diagram



1. Purpose

This state chart diagram details the life cycle, operational states, and behavioral transitions within the Academic Feedback & Improvement Management System (AFIMS). It illustrates how different user roles (Students, Faculty, HOD, and Admin) navigate the system from authentication through core operational states to final session closure.

2. Primary States and Sub-States

- **Authentication State (Login):** The entry point where credentials are validated. It contains an self-transition loop for handling invalid credentials.
- **Navigation Center (Dashboard):** The central state that dynamically displays role-based options upon a valid login.
- **Functional Domain States:**
 - **Feedback:** Student-facing state for answering questions, rating, and submitting evaluations.
 - **Question Management & Review:** Faculty and HOD-facing states covering the lifecycle of a question (Creation → Pending Approval → Approved/Rejected).
 - **User Management:** Admin-facing state for user CRUD operations and role assignment.

- **System Configuration:** Admin-facing state for adjusting core system preferences.
- **Termination State (Logout):** Clears the active session and exits the system lifecycle.

B.5.UML Component Diagram:

The **AFIMS (Academic Feedback and Improvement Management System)** component diagram illustrates the high-level modular architecture of the system and the interactions among its major software components. The diagram represents how the system is divided into independent functional modules and how these modules communicate through defined interfaces.

The component diagram provides a structural view of AFIMS by identifying the major components responsible for authentication, user management, survey administration, question management, feedback collection, improvement planning, suggestions, and reporting and analytics. It also represents the system's interaction with the underlying **MySQL database** for persistent data storage.

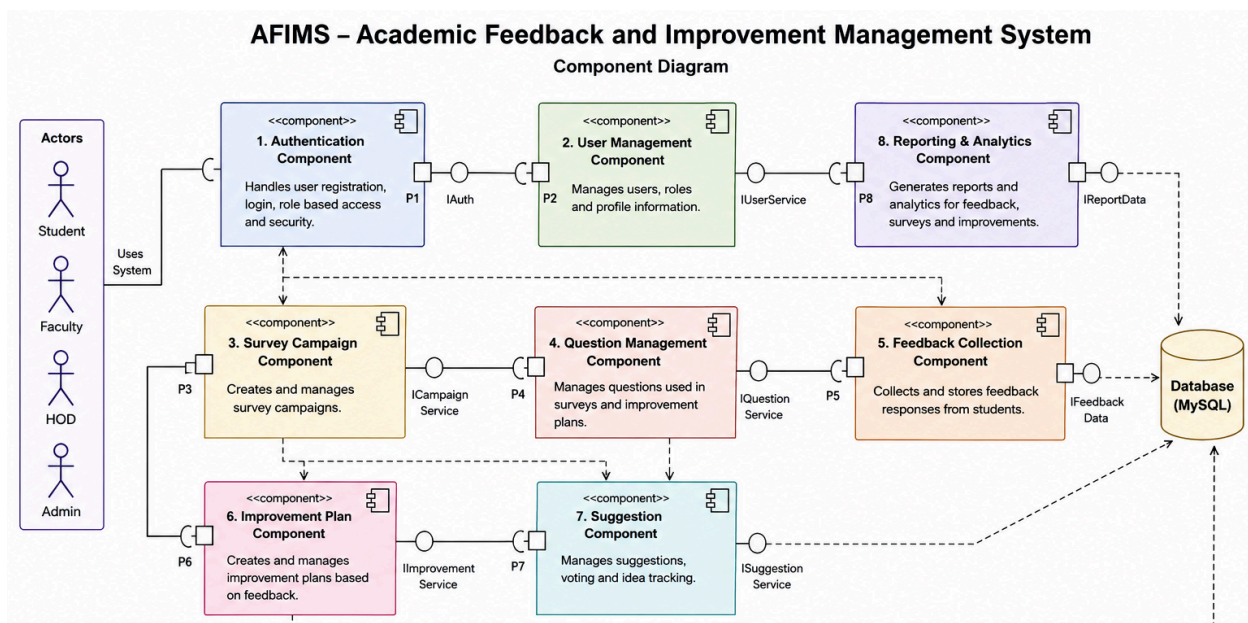


Figure A.1: UML Component Diagram of AFIMS

B.5.1 Components of the System

The major components represented in the AFIMS component diagram are described below:

Component	Description
Authentication Component	Handles user registration, login, authentication, and role-based access control for Students, Faculty, HOD, and Admin.
User Management Component	Manages user profiles, roles, and user-related information within the system.
Survey Campaign Component	Enables the HOD to create, manage, and control feedback survey campaigns.
Question Management Component	Manages the questions used in feedback surveys and improvement-related activities.
Feedback Collection Component	Collects and processes feedback responses submitted by students.
Improvement Plan Component	Supports the creation and management of improvement plans based on feedback results and

Component	Description
	identified areas of improvement.
Suggestion Component	<i>Manages</i> student suggestions, including submission, viewing, and voting mechanisms.
Reporting & Analytics Component	Generates reports and analytical information from feedback, survey, suggestion, and improvement -plan data.
MySQL Database	Provides persistent storage for user information, survey campaigns, questions, feedback responses, suggestions, improvement plans, and analytical data.

B.5.2 Actors and Component Interaction

The system supports four primary user roles:

- **Student** — Participates in surveys, submits feedback, posts suggestions, and votes on suggestions.
- **Faculty** — *Manages* survey questions, views feedback, and prepares improvement plans.
- **HOD** — Creates and manages survey campaigns, approves questions, manages suggestions, and reviews analytical information.
- **Admin** — *Manages* users, configures the system, maintains records, and performs administrative operations.

These actors interact with the appropriate system components through the application's defined interfaces.

B.6.UML Deployment Diagram:

The UML Deployment Diagram of the AFIMS (*Academic Feedback and Improvement Management System*) illustrates the physical deployment of the system's software components across different computational nodes. It represents the hardware and server environment required to execute the AFIMS application and shows the communication pathways between the deployed components.

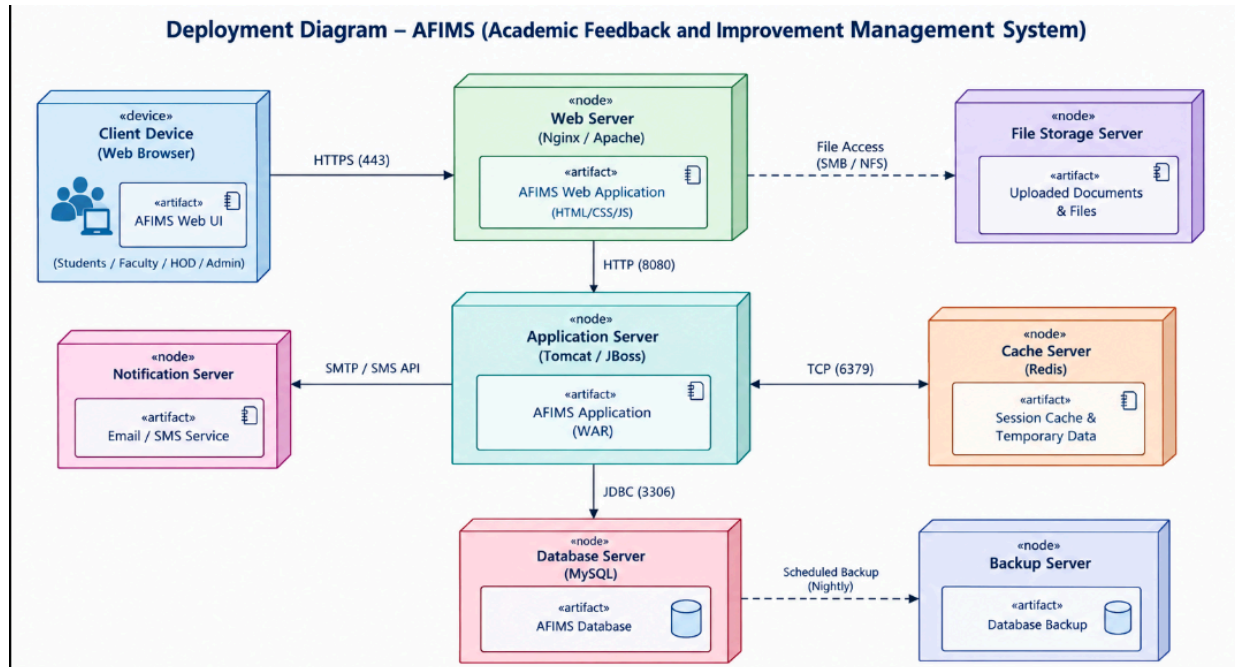


Figure B.6.1: UML Deployment Diagram of AFIMS

B.6.1 Deployment Modes

The major deployment nodes of AFIMS are described below.

Deployment Mode	Description
Client Device	Represents the user's computer or device running a web browser. Students, Faculty, HOD, and Admin access AFIMS through the web interface.
Web Server	Hosts the AFIMS web application and handles incoming HTTPS requests from client devices. A web server such as Nginx or Apache may be used.
Application Server	Executes the core AFIMS application and business logic. An application server such as Tomcat or JBoss may be used for hosting the application.
Database Server	Hosts the <i>MySQL</i> database containing persistent AFIMS data such as users, survey campaigns, questions, feedback responses, suggestions, and improvement plans.
File Storage Server	Stores uploaded documents and other files required by the application. File access may be provided through mechanisms such as <i>SMB</i> or <i>NFS</i> .
Cache Server	Provides temporary storage for frequently accessed data and session information. Redis may be used to improve response time and application performance.

B.7.IMPLEMENTATION:

B.7.1.Frontend:

Home.html

```
<!doctype html>

<html lang="en" xmlns:th="http://www.thymeleaf.org">

<head th:replace="~{fragments/head :: head(AFIMS — Unleash Better Learning)}">

  <meta charset="UTF-8">

  <title>AFIMS</title>

</head>

<body class="landing">

  <div class="energy"></div>

  <header class="landing-nav">

    <a class="brand" th:href="@{/}">AFIMS <span>◆</span></a>

    <div>

      <a th:href="@{/login}" class="btn btn-ghost">Login</a>

      <a th:href="@{/register}" class="btn btn-primary">Register</a>

    </div>

  </header>

  <main>

    <!-- Hero Section -->

    <section class="hero container">

      <div class="eyebrow">GAMMA ACADEMY · ACADEMIC INTELLIGENCE</div>

      <h1>Unleash Better<br><span>Learning.</span></h1>

      <p class="lead">AFIMS turns structured student feedback into clear academic insights, accountable improvement plans, and measurable follow-through.</p>

      <div class="hero-actions">

        <a th:href="@{/login}" class="btn btn-primary btn-lg">Enter AFIMS</a>

        <a th:href="@{/register}" class="btn btn-ghost btn-lg">Create account</a>

      </div>

      <div class="hero-rule">

        <span></span>

        <small>STRONG FEEDBACK · STRONGER LEARNING</small>

        <span></span>

      </div>

    </section>

    <!-- Process Workflow Section -->

    <section class="container section">

      <div class="section-title">

        <span>01</span>

        <h2>How AFIMS Works</h2>

      </div>

      <div class="steps">

        <article>

          <b>01</b>

          <h3>Collect</h3>

          <p>Capture structured faculty feedback during active academic cycles.</p>

        </article>
```

```
      <article>

        <b>02</b>

        <h3>Understand</h3>

        <p>Surface trends, themes, participation and actionable feedback signals.</p>

      </article>

      <article>

        <b>03</b>

        <h3>Improve</h3>

        <p>Convert evidence into faculty-owned improvement plans for HOD review.</p>

      </article>

      <article>

        <b>04</b>

        <h3>Track</h3>

        <p>Follow approvals, implementation and future feedback outcomes.</p>

      </article>

    </div>

  </section>

  <!-- Features Feature-Grid Section -->

  <section class="container section">

    <div class="feature-grid">

      <div>

        <div class="section-title">

          <span>02</span>

          <h2>Built for academic teams</h2>

        </div>

        <p class="muted big">A role-aware workspace for students, faculty, HODs and administrators — with a visual identity that feels like a product, not a classroom template.</p>

      </div>

      <div class="feature-list">

        <div>

          <strong>Smart Feedback</strong>

          <span>Configurable 1–5 academic questions.</span>

        </div>

        <div>

          <strong>Gamma Insights</strong>

          <span>Local MLP-style analysis without paid APIs.</span>

        </div>

        <div>

          <strong>Improvement Plans</strong>

          <span>Draft → review → approve → implement.</span>

        </div>

        <div>

          <strong>Academic Analytics</strong>

          <span>Clear factual trends using Chart.js-ready data.</span>

        </div>

      </div>

    </div>
```

```

        </div>
    </section>
</main>

<footer th:replace="~{fragments/footer :: footer}"></footer>
</body>
</html>

```

Notification.html

```

<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head('Notifications' · AFIMS)}"></head>
<body>

    <div th:replace="~{fragments/nav :: nav}"></div>

    <main class="container page">
        <div class="page-head">
            <div>
                <div class="eyebrow">ACADEMIC SIGNALS</div>
                <h1>Notifications</h1>
                <p class="muted">Updates connected to your AFIMS account.</p>
            </div>
        </div>

        <section class="panel">
            <!-- Empty state when there are no notifications -->
            <div th:if="${#lists.isEmpty(items)}" class="empty">
                No notifications yet.
            </div>

            <!-- Loop through each notification item -->
            <div class="plan-mini" th:each="n : ${items}">
                <div>
                    <b th:text="${n.title}"></b>
                    <span th:text="${n.message}"></span>
                    <small th:text="${n.createdAt}"></small>
                </div>

                <!-- Mark as read button (only displays if readFlag is false) -->
                <form th:if="${n.readFlag}" th:action="@{/notifications/ · ${n.id} · /read}"
method="post">
                    <button class="btn btn-small">Mark read</button>
                </form>
            </div>
        </section>
    </main>

</body>

```

```
</html>
```

Student/dashboard.html

```

<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head('Student Dashboard' · AFIMS)}"></head>
<body>

    <div th:replace="~{fragments/nav :: nav}"></div>

    <main class="container page">
        <!-- Dashboard Header -->
        <div class="page-head">
            <div>
                <div class="eyebrow">STUDENT WORKSPACE</div>
                <h1>Welcome, <span th:text="${user.fullName}">Student</span></h1>
                <p class="muted">Your feedback helps strengthen the learning
experience.</p>
            </div>
            <a href="/student/feedback/new" class="btn btn-primary">Give Feedback</a>
        </div>

        <!-- Metrics / Statistics -->
        <div class="stats">
            <div class="stat">
                <span>Feedback submitted</span>
                <strong th:text="${feedbacks.size()}">0</strong>
            </div>
            <div class="stat">
                <span>Feedback submissions</span>
                <strong th:text="${feedbacks.size()}">0</strong>
            </div>
            <div class="stat">
                <span>Active cycle</span>
                <strong th:text="${activeCycle != null ? 'OPEN' : 'CLOSED'}">OPEN</strong>
            </div>
        </div>

        <!-- Main Layout Grid -->
        <div class="grid-2">

            <!-- Left Column: Recent Feedback Table -->
            <section class="panel">
                <div class="panel-head">
                    <h2>Recent Feedback</h2>
                    <span class="tag">HISTORY</span>
                </div>

```

```

<div th:if="${#lists.isEmpty(feedbacks)}" class="empty">
    No feedback submitted yet.
</div>

<div class="table-wrap" th:if="${#lists.isEmpty(feedbacks)}">
    <table>
        <thead>
            <tr>
                <th>Faculty</th>
                <th>Course</th>
                <th>Rating</th>
                <th>Date</th>
            </tr>
        </thead>
        <tbody>
            <tr th:each="f : ${feedbacks}">
                <td th:text="${f.anonymous ? 'Anonymous' :
f.faculty.fullName}"></td>
                <td th:text="${f.course}"></td>
                <td><span class="rating" th:text="${f.overallRating *
'/5}'></span></td>
                <td th:text="${#temporals.format(f.createdAt, 'dd MMM
yyyy)}"></td>
            </tr>
        </tbody>
    </table>
</div>
</section>

<!-- Right Column: Improvement Tracker -->
<section class="panel insight-panel">
    <div class="panel-head">
        <h2>Improvement Tracker</h2>
        <span class="gamma-badge">AFIMS</span>
    </div>

    <div class="tracker">
        <div>
            <b>Feedback cycle</b>
            <span>Active cycles let students contribute without duplicate
submissions.</span>
        </div>
        <div>
            <b>Department response</b>
            <span>Improvement updates appear here when plans are approved
and implemented.</span>
        </div>
        <div>
            <b>Notifications</b>
            <span>Stay informed about feedback cycles and academic

```

```

improvements.</span>
        </div>
    </div>
</section>

</div>
</main>

<footer th:replace="~{fragments/footer :: footer}"></footer>

</body>
</html>

Student/feedback.html
<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head(Submit Feedback · AFIMS)}"></head>
<body>

    <div th:replace="~{fragments/nav :: nav}"></div>

    <main class="container page narrow">
        <!-- Page Header -->
        <div class="page-head">
            <div>
                <div class="eyebrow">FEEDBACK CYCLE</div>
                <h1>Strengthen the classroom.</h1>
                <p class="muted">Rate the learning experience honestly and
constructively.</p>
            </div>
        </div>

        <!-- Alert Notifications -->
        <div th:if="${error}" class="alert alert-danger" th:text="${error}"></div>

        <div th:if="${activeCycle == null}" class="alert alert-warning">
            There is no active feedback cycle. Please return later.
        </div>

        <!-- Feedback Submission Form -->
        <form th:if="${activeCycle != null}" th:action="@{/student/feedback}"
th:object="${form}" method="post" class="panel form-card">

            <!-- Inputs Grid Configuration -->
            <div class="form-grid">
                <label>Faculty
                <select th:field="*{facultyId}" required>
                    <option value="">Select faculty</option>
                    <option th:each="f : ${faculties}" th:value="${f.id}"
th:text="${f.fullName}"></option>

```

```

        </select>

</label>

<label>Course
    <input th:field="*{course}" placeholder="e.g. Data Science" required>
</label>

<label>Subject
    <input th:field="*{subject}" placeholder="e.g. Unit 3" required>
</label>

<label>Overall rating
    <select th:field="*{rating}" required>
        <option value="">Choose 1–5</option>
        <option value="1">1 — Needs attention</option>
        <option value="2">2</option>
        <option value="3">3 — Adequate</option>
        <option value="4">4</option>
        <option value="5">5 — Strong</option>
    </select>
</label>
</div>

<!-- Dynamic Question List Display -->
<div class="question-list">
    <div th:each="q : ${questions}" class="question-row">
        <span th:text="${q.questionText}"></span>
        <span class="tag" th:text="${q.category}"></span>
    </div>
</div>

<!-- Long Text Inputs & Flags -->
<label>Positive feedback / comment
    <textarea th:field="*{comment}" rows="4" placeholder="What worked well?"></textarea>
</label>

<label>Improvement suggestion
    <textarea th:field="*{suggestion}" rows="4" placeholder="What could make the learning experience stronger?"></textarea>
</label>

<label class="check">
    <input th:field="*{anonymous}" type="checkbox"> Submit as anonymous where institution policy permits
</label>

<!-- Action Button -->
<button class="btn btn-primary" type="submit">Submit Feedback</button>

</form>

```

```

</main>

</body>
</html>

Hod/dashboard.html:
<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head(HOD Dashboard · AFIMS)}"></head>
<body>

    <div th:replace="~{fragments/nav :: nav}"></div>

    <main class="container page">

        <!-- Page Header -->
        <div class="page-head">
            <div>
                <div class="eyebrow">DEPARTMENT OVERSIGHT</div>
                <h1>Review & improve.</h1>
                <p class="muted">Monitor academic feedback and the implementation of faculty improvement plans.</p>
            </div>
        </div>

        <!-- Metrics / Statistics -->
        <div class="stats">
            <div class="stat">
                <span>Faculty</span>
                <strong th:text="${facultyCount}">0</strong>
            </div>
            <div class="stat">
                <span>Students</span>
                <strong th:text="${studentCount}">0</strong>
            </div>
            <div class="stat">
                <span>Average rating</span>
                <strong th:text="${#numbers.formatDecimal(avg.12)}">0.00</strong>
            </div>
        </div>

        <!-- Improvement Plan Table Section -->
        <section class="panel">
            <div class="panel-head">
                <h2>Improvement Plan Review</h2>
                <span class="tag">FACTUAL WORKFLOW</span>
            </div>

            <div class="table-wrap">
                <table>
                    <thead>

```

```

<tr>
  <th>Faculty</th>
  <th>Plan</th>
  <th>Status</th>
  <th>Review</th>
</tr>
</thead>
<tbody>
<tr th:each="p : ${plans}">
  <td th:text="${p.faculty.fullName}"></td>
  <td th:text="${p.title}"></td>
  <td>
    <span class="status" th:text="${p.status}"></span>
  </td>
  <td>
    <form method="post" th:action="@{/hod/plans/ * ${p.id} +
'/review}" class="review-form">
      <select name="decision">
        <option value="UMDER_REVIEW">Under review</option>
        <option value="APPROVED">Approve</option>
        <option value="CHANGES_REQUESTED">Request
changes</option>
        <option value="REJECTED">Reject</option>
      </select>
      <input name="comments" placeholder="HOD comment">
      <button class="btn btn-small">Save</button>
    </form>
  </td>
</tr>
</tbody>
</table>
</div>
</section>
</main>

</body>
</html>

```

Faculty/dashboard.html:

```

<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head(Faculty Dashboard - AFIMS)}"></head>
<body>

  <div th:replace="~{fragments/nav :: nav}"></div>

  <main class="container page">
    <!-- Page Header -->
    <div class="page-head">
      <div>

```

```

        <div class="eyebrow">FACULTY WORKSPACE</div>
        <h1>Your learning signals.</h1>
        <p class="muted">Review feedback, identify improvement areas and own
the response.</p>
      </div>
      <a href="/faculty/plans/new" class="btn btn-primary">Create Improvement
Plan</a>
    </div>

    <!-- Metrics / Statistics -->
    <div class="stats">
      <div class="stat">
        <span>Overall rating</span>
        <strong th:text="${#numbers.formatDecimal(avg.1.2)}">0.00</strong>
      </div>
      <div class="stat">
        <span>Responses</span>
        <strong th:text="${feedbacks.size()}">0</strong>
      </div>
      <div class="stat">
        <span>Plans</span>
        <strong th:text="${plans.size()}">0</strong>
      </div>
    </div>

    <!-- Main Layout Grid -->
    <div class="grid-2">

      <!-- Left Column: Recent Feedback Table -->
      <section class="panel">
        <div class="panel-head">
          <h2>Recent Feedback</h2>
          <a href="/faculty/insights">Gamma Insights → </a>
        </div>

        <div class="table-wrap">
          <table>
            <thead>
              <tr>
                <th>Course</th>
                <th>Rating</th>
                <th>Comment</th>
              </tr>
            </thead>
            <tbody>
              <tr th:each="f : ${feedbacks}">
                <td th:text="${f.course}"></td>
                <td>
                  <span class="rating" th:text="${f.overallRating * '1/5'}"></span>
                </td>

```

```

        <td th:text='${f.comment != null ? f.comment : ''}'></td>
    </tr>
</tbody>
</table>
</div>
</section>

<!-- Right Column: Improvement Plans Workflow -->
<section class="panel">
    <div class="panel-head">
        <h2>Improvement Plans</h2>
        <span class="tag">WORKFLOW</span>
    </div>

    <div th:if='${#lists.isEmpty(plans)}' class="empty">
        No plans yet
    </div>

    <div class="plan-mini" th:each="p : ${plans}">
        <div>
            <b th:text='${p.title}'></b>
            <span th:text='${p.status}'></span>
        </div>

        <form th:if='${p.status.name() == 'DRAFT' || p.status.name() == 'CHANGES_REQUESTED'}' th:action="@{/faculty/plans/ * ${p.id} * /submit}" method="post">
            <button class="btn btn-small">Submit to HOD</button>
        </form>
    </div>
</section>

</div>

<!-- Full-Width Bottom Section: Question Management -->
<section class="panel">
    <div class="panel-head">
        <h2>Question Management</h2>
        <span class="tag">FACULTY CONFIG</span>
    </div>

    <!-- Form to Add a New Question -->
    <form method="post" action="/faculty/questions" class="form-grid">
        <label>Question
            <input name="questionText" required placeholder="Add a learning-experience question">
        </label>
        <label>Category
            <input name="category" required placeholder="Teaching Quality">
        </label>
        <button class="btn btn-primary">Add question</button>
    </form>
</div>

```

```

</form>

<!-- List of Existing Questions -->
<div class="question-list compact">
    <div th:each="q : ${questions}">
        <span th:text='${q.questionText}'></span>
        <span>
            <span class="tag" th:text='${q.category}'></span>

            <form class="inline" style="display:inline" method="post"
                th:action="@{/faculty/questions/ * ${q.id} * /disable}">
                <button class="btn btn-small" type="submit">Disable</button>
            </form>
        </span>
    </div>
</div>
</section>
</main>

</body>
</html>

```

Faculty/insights.html:

```

<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head(Gamma Insights - AFIMS)}"></head>
<body>

    <div th:replace="~{fragments/nav :: nav}"></div>

    <main class="container page">
        <!-- Page Header -->
        <div class="page-head">
            <div>
                <div class="eyebrow">LOCAL MLP-STYLE ANALYSIS</div>
                <h1>Gamma Insight Assistant</h1>
                <p class="muted">A transparent, rule-based first layer. Review the evidence before acting on it.</p>
            </div>
        </div>

        <!-- Insights Container Grid -->
        <div class="insight-grid">

            <!-- Loop through feedbacks using an index tracker 'i' for matched analysis lookups -->
            <article class="panel" th:each="f, i : ${feedbacks}">

                <div class="panel-head">
                    <h2 th:text='${Feedback # * f.id}'></h2>
                    <span class="rating" th:text='${f.overallRating * '1/5'}"></span>
                </div>
            </article>
        </div>
    </main>
</body>
</html>

```

```

</div>

<p th:text="{f.comment}"></p>

<!-- Positive Themes Chips Wrapper -->
<div th:if="{#lists.isEmpty(analyses[i.index].positiveThemes)}">
    <b>Positive themes</b>
    <div class="chips">
        <span th:each="x : {analyses[i.index].positiveThemes}"
th:text="{x}"></span>
    </div>
</div>

<!-- Improvement Themes Chips Wrapper -->
<div>
    <b>Improvement themes</b>
    <div class="chips">
        <span th:each="x : {analyses[i.index].improvementThemes}"
th:text="{x}"></span>
        <span
th:if="{#lists.isEmpty(analyses[i.index].improvementThemes)}">No recurring theme
detected</span>
    </div>
</div>

<!-- Recommendation block based on rule processing -->
<p class="recommendation">
    <b th:text="{Priority . . . analyses[i.index].priority}"></b>
    <br>
    <span th:text="{analyses[i.index].recommendation}"></span>
</p>

</article>

</div>
</main>

</body>
</html>

```

Faculty/plan-form.html:

```

<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="{fragments/head :: head(Improvement Plan - AFIMS)}"></head>
<body>

    <div th:replace="{fragments/nav :: nav}"></div>

    <main class="container page narrow">
        <!-- Page Header -->
        <div class="page-head">

```

```

<div>
    <div class="eyebrow">GAMMA INSIGHT ASSISTANT</div>
    <h1>Build an improvement plan.</h1>
    <p class="muted">Turn evidence into an accountable action with a
measurable outcome.</p>
</div>
</div>

<!-- Improvement Plan Creation Form -->
<form th:action="{@{/faculty/plans}}" th:object="{form}" method="post"
class="panel form-card">

    <label>Title
    <input th:field="{title}" placeholder="e.g. Increase practical demonstrations"
required>
</label>

    <label>Problem statement
    <textarea th:field="{problemStatement}" required></textarea>
</label>

    <label>Evidence
    <textarea th:field="{evidence}" placeholder="Repeated feedback, category
trends, observations..."></textarea>
</label>

    <label>Root cause
    <textarea th:field="{rootCause}"></textarea>
</label>

    <label>Proposed action
    <textarea th:field="{proposedAction}" required></textarea>
</label>

<!-- Contextual Grid Settings -->
<div class="form-grid">
    <label>Target date
    <input th:field="{targetDate}" type="date">
</label>

    <label>Priority
    <select th:field="{priority}">
        <option>Low</option>
        <option>Medium</option>
        <option>High</option>
    </select>
</label>
</div>

    <label>Expected outcome
    <textarea th:field="{expectedOutcome}"></textarea>

```

```

</label>

<label>Measurement method
  <textarea th:field="*{measurement/Method}" placeholder="How will you
know the action worked?"></textarea>
</label>

<!-- Action Button -->
<button class="btn btn-primary" type="submit">Save Draft</button>

</form>
</main>

</body>
</html>

```

Admin/audit.html:

```

<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head(Audit Log · AFIMS)}"></head>
<body>

<div th:replace="~{fragments/nav :: nav}"></div>

<main class="container page">
  <!-- Page Header -->
  <div class="page-head">
    <div>
      <div class="eyebrow">SYSTEM ACCOUNTABILITY</div>
      <h1>Audit log.</h1>
      <p class="muted">Recent security and workflow events.</p>
    </div>
  </div>

  <!-- Audit Log Table Section -->
  <section class="panel">
    <div class="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Timestamp</th>
            <th>User</th>
            <th>Action</th>
            <th>Entity</th>
            <th>IP</th>
          </tr>
        </thead>
        <tbody>
          <!-- Loop through the pageable log content collection -->
          <tr th:each="l : ${logs.content}">

```

```

      <td th:text="${l.timestamp}"></td>
      <td th:text="${l.user != null ? l.user.email : 'SYSTEM'}"></td>
      <td th:text="${l.action}"></td>
    </td>
    <td th:text="${l.ipAddress != null ? l.ipAddress : '-'}"></td>
  </tr>
</tbody>
</table>
</div>
</section>
</main>

</body>
</html>

```

Admin/dashboard.html:

```

<!doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/head :: head(Admin Console · AFIMS)}"></head>
<body>

<div th:replace="~{fragments/nav :: nav}"></div>

<main class="container page">
  <!-- Page Header -->
  <div class="page-head">
    <div>
      <div class="eyebrow">ADMINISTRATION</div>
      <h1>AFIMS control room.</h1>
      <p class="muted">Manage the academic feedback ecosystem and keep
the workflow accountable.</p>
    </div>
    <a href="/admin/audit" class="btn btn-ghost">View Audit Log</a>
  </div>

  <!-- Metrics Dashboard Grid -->
  <div class="stats">
    <div class="stat">
      <span>Total users</span>
      <strong th:text="${totalUsers}">0</strong>
    </div>
    <div class="stat">
      <span>Active users</span>
      <strong th:text="${activeUsers}">0</strong>
    </div>
    <div class="stat">
      <span>Pending registrations</span>
      <strong th:text="${pending}">0</strong>
    </div>
    <div class="stat">

```

```

        <span>Feedback submissions</span>
        <strong th:text="{feedbackCount}">0</strong>
    </div>
    <div class="stat">
        <span>Average rating</span>
        <strong th:text="{#numbers.formatDecimal(avg,1,2)}">0.00</strong>
    </div>
    <div class="stat">
        <span>Open plans</span>
        <strong th:text="{openPlans}">0</strong>
    </div>
</div>

<!-- Split Grid Section -->
<div class="grid-2">

    <!-- Left Side: User Access & Approvals Table -->
    <section class="panel">
        <div class="panel-head">
            <h2>User approvals</h2>
            <span class="tag">ACCESS</span>
        </div>

        <div class="table-wrap">
            <table>
                <thead>
                    <tr>
                        <th>User</th>
                        <th>Role</th>
                        <th>Status</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody>
                    <tr th:each="u : {users.content}">
                        <td>
                            <b th:text="{u.fullName}"></b>
                            <small th:text="{u.email}"></small>
                        </td>
                        <td th:text="{u.role}"></td>
                        <td th:text="{u.status}"></td>
                        <td>
                            <form method="post" th:action="@{/admin/users/} + {u.id} +
                                /status}" class="inline">
                                <select name="value">
                                    <option value="ACTIVE">Activate</option>
                                    <option value="REJECTED">Reject</option>
                                    <option value="SUSPENDED">Suspend</option>
                                </select>
                                <button class="btn btn-small">Apply</button>
                            </form>
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
    </section>

    <!-- Right Side: Question Bank Management -->
    <section class="panel">
        <div class="panel-head">
            <h2>Question bank</h2>
            <span class="tag">CONFIG</span>
        </div>

        <form method="post" action="/admin/questions" class="stack">
            <input name="questionText" placeholder="New feedback question"
                required>
            <input name="category" placeholder="Category" required>
            <button class="btn btn-primary">Add question</button>
        </form>

        <div class="question-list compact">
            <div th:each="q : {questions}">
                <span th:text="{q.questionText}"></span>
                <span class="tag" th:text="{q.category}"></span>
            </div>
        </div>
    </section>

</div>

<!-- Bottom Full-Width Section: Feedback Cycles Management -->
<section class="panel">
    <div class="panel-head">
        <h2>Feedback cycles</h2>
        <span class="tag">ACADEMIC CALENDAR</span>
    </div>

    <form method="post" action="/admin/cycles" class="cycle-form">
        <input name="name" placeholder="Cycle name" required>
        <input name="academicYear" placeholder="2026-27" required>
        <input name="semester" type="number" min="1" max="2"
            placeholder="Semester" required>
        <input name="startDate" type="date" required>
        <input name="endDate" type="date" required>
        <button class="btn btn-primary">Create cycle</button>
    </form>

    <div class="chips">

```

```

        <span th:each="c : ${cycles}" th:text="${c.name} ··· · c.status"></span>
    </div>
</section>
</main>

</body>
</html>

Auth/login.html:
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">

<head th:replace="~{fragments/head :: head(Login · AFIMS)}"></head>

<body class="auth-page">

<div class="auth-shell">

<a href="/" class="brand">
    AFIMS <span>◆</span>
</a>

<div class="auth-card">

<div class="eyebrow">SECURE ACADEMIC ACCESS</div>

<h1>Welcome back.</h1>

<p class="muted">
    Sign in to your AFIMS workspace.
</p>

<div th:if="${param.error}" class="alert alert-danger">
    Email or password is incorrect, or the account is not active.
</div>

<div th:if="${param.logout}" class="alert alert-success">
    You have been signed out.
</div>

<div th:if="${param.forgot}" class="alert alert-warning">
    Password reset is prepared as a UI workflow.
    Connect an institutional mail provider before enabling
    production resets.
</div>

<div th:if="${success}" class="alert alert-success" th:text="${success}"></div>

<form th:action="@{/login}" method="post">

```

```

<label>
    Email
    <input
        name="username"
        type="email"
        required
        autocomplete="email">
</label>

<label>
    Password
    <input
        name="password"
        type="password"
        required
        autocomplete="current-password">
</label>

<div class="form-row">

<label class="check">
    <input type="checkbox" name="remember-me">
    Remember me
</label>

<a href="/login?forgot=!" class="muted">
    Forgot password?
</a>

</div>

<button
    class="btn btn-primary w-100"
    type="submit">
    Login to AFIMS
</button>

</form>

<p class="auth-bottom">
    New to AFIMS?
    <a href="/register">Create an account</a>
</p>

</div>

</body>
</html>

```

Auth/register.html:

```
<!doctype html>

<html xmlns:th="http://www.thymeleaf.org">

<head th:replace="~{fragments/head :: head(Register · AFIMS)}"></head>

<body class="auth-page">

    <div class="auth-shell wide">

        <!-- Brand Link -->

        <a href="/" class="brand">AFIMS <span>◆</span></a>

        <!-- Registration Card Container -->

        <div class="auth-card">

            <div class="eyebrow">ACCOUNT REQUEST</div>

            <h1>Join AFIMS.</h1>

            <p class="muted">Create an institutional account. New registrations remain pending until approved by an administrator.</p>

            <!-- Global / Action Error Banner -->

            <div th:if="${error}" class="alert alert-danger" th:text="${error}"></div>

            <!-- Multi-Section Request Form -->

            <form th:action="@{/register}" th:object="${form}" method="post">

                <!-- Section 1: Personal Information -->

                <div class="form-section">

                    <h3>Personal Information</h3>

                    <div class="form-grid">

                        <label>Full Name

                            <input th:field="*{fullName}" required>

                            <small th:errors="*{fullName}"></small>

                        </label>

                        <label>Email

                            <input th:field="*{email}" type="email" required>

                            <small th:errors="*{email}"></small>

                        </label>

                        <label>Phone

                            <input th:field="*{phone}" required>

                        </label>

                        <label>Role

                            <select th:field="*{role}" required>

                                <option value="">Select role</option>

                                <option value="STUDENT">Student</option>

                                <option value="FACULTY">Faculty</option>

                                <option value="HOD">HOD</option>

                            </select>

                        </label>

                    </div>

                </div>

            </form>

        </div>

    </div>
```

```


    </div>

</div>

<!-- Section 2: Institution Details -->

<div class="form-section">

    <h3>Institution Information</h3>

    <div class="form-grid">

        <label>Institution

            <input th:field="*{institution}" required>

        </label>

        <label>Department

            <input th:field="*{department}" required>

        </label>

        <label>Institution ID

            <input th:field="*{institutionId}" required>

        </label>

    </div>

</div>

<!-- Section 3: Account Security -->

<div class="form-section">

    <h3>Account Security</h3>

    <div class="form-grid">

        <label>Password

            <input th:field="*{password}" id="password" type="password" minlength="8" required>

            <!-- Password Strength Visual Indicator -->

            <div class="strength">

                <div></div><div></div><div></div><div></div><div></div><div></div>

            </div>

        </label>

        <label>Confirm Password

            <input th:field="*{confirmPassword}" type="password" required>

        </label>

    </div>

</div>

<!-- Submit Action -->

<button class="btn btn-primary w-100" type="submit">Submit Registration</button>

</form>

<!-- Footer Utilities -->

<p class="auth-bottom">Already registered? <a href="/login">Return to login</a></p>

</div>

</div>
```

```

</body>
</html>

```

Afims.js:

```

document.addEventListener('DOMContentLoaded', () => {
    const p = document.querySelector('#password');
    if (p) {
        const bars = [...document.querySelectorAll('.strength i')];
        p.addEventListener('input', () => {

```

```

        let s = 0;
        if (p.value.length >= 8) s++;
        if (/[A-Z]/.test(p.value) && /[a-z]/.test(p.value)) s++;
        if (/^d/.test(p.value)) s++;
        if (/^[A-Za-z0-9]/.test(p.value)) s++;
        bars.forEach((b, i) => b.classList.toggle('on', i < s));
    });
}
});

```

B.7.2.Backend:

AFIMSApplication.java

```

package com.afims;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class AFIMSApplication {
    public static void main(String[] args) {
        SpringApplication.run(AFIMSApplication.class, args);
    }
}

```

AdminController.java:

```

package com.afims.controller;
import org.springframework.data.domain.PageRequest;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import com.afims.entity.FeedbackCycle;
import com.afims.entity.ImprovementPlan;
import com.afims.entity.Question;
import com.afims.entity.User;
import com.afims.repository.AuditLogRepository;
import com.afims.repository.DepartmentRepository;
import com.afims.repository.FeedbackCycleRepository;
import com.afims.repository.FeedbackRepository;
import com.afims.repository.ImprovementPlanRepository;
import com.afims.repository.QuestionRepository;
import com.afims.repository.UserRepository;
@Controller
@RequestMapping("/admin")public class AdminController {
    final UserRepository users;
    final FeedbackRepository feedbacks;
    final ImprovementPlanRepository plans;
    final AuditLogRepository audits;
    final FeedbackCycleRepository cycles;
    final QuestionRepository questions;
    final DepartmentRepository depts;
    public AdminController(
        UserRepository u, FeedbackRepository f,
        ImprovementPlanRepository p, AuditLogRepository a,

```

```

        FeedbackCycleRepository c,
        QuestionRepository q,
        DepartmentRepository d) {
        users = u;
        feedbacks = f;
        plans = p;
        audits = a;
        cycles = c;
        questions = q;
        depts = d;
    }
    @GetMapping("/dashboard") String dash(Model m) {
        m.addAttribute("users", users.findAll(PageRequest.of(0, 10)));
        m.addAttribute("totalUsers", users.count());
        m.addAttribute("activeUsers", users.countByStatus(
            User.RegistrationStatus.ACTIVE));
        m.addAttribute("pending", users.countByStatus(
            User.RegistrationStatus.PENDING));
        m.addAttribute("feedbackCount", feedbacks.total());
        m.addAttribute("avg", feedbacks.averageRating() == null ? 0 : feedbacks.averageRating());
        m.addAttribute("openPlans", plans.countByStatus(
            ImprovementPlan.Status.SUBMITTED));
        m.addAttribute("cycles", cycles.findAll());
        m.addAttribute("questions", questions.findAll());
        return "admin/dashboard";
    }
    @PostMapping("/users/{id}/status") String status(
        @PathVariable Long id,
        @RequestParam User.RegistrationStatus value) {
        User u = users.findById(id).orElseThrow();
        u.setStatus(value);
        users.save(u);
        return "redirect:/admin/dashboard";
    }
    @PostMapping("/cycles") String cycle(
        @RequestParam String name,
        @RequestParam String academicYear,
        @RequestParam Integer semester,
        @RequestParam java.time.LocalDate startDate,
        @RequestParam java.time.LocalDate endDate) {

```

```

        FeedbackCycle c = new FeedbackCycle();
        c.setName(name);
        c.setAcademicYear(academicYear);
        c.setSemester(semester);
        c.setStartDate(startDate);
        c.setEndDate(endDate);
        c.setStatus(FeedbackCycle.Status.UPCOMING);
        cycles.save(c);
        return "redirect:/admin/dashboard";
    }
    @PostMapping("/questions") String question(
    @RequestParam String questionText,
    @RequestParam String category) {
        Question q = new Question();
        q.setQuestionText(questionText);
        q.setCategory(category);
        q.setDisplayOrder((int) questions.count() + 1);
        questions.save(q);
        return "redirect:/admin/dashboard";
    }
    @GetMapping("/audit") String audit(Model m) {
        m.addAttribute(
            "logs", audits.findAll(PageRequest.of(0,
50))
        );
        return "admin/audit";
    }
}

```

FacultyController.java:

```

package com.afims.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import com.afims.dto.ImprovementPlanRequest;
import com.afims.entity.Feedback;
import com.afims.entity.ImprovementPlan;
import com.afims.entity.Question;
import com.afims.entity.User;
import com.afims.repository.AuditLogRepository;
import com.afims.repository.FeedbackRepository;
import com.afims.repository.ImprovementPlanRepository;
import com.afims.repository.QuestionRepository;
import com.afims.repository.UserRepository;
import com.afims.service.FeedbackAnalysisService;
import com.afims.service.NotificationService;
import jakarta.validation.Valid;

@Controller
@RequestMapping("/faculty") public class FacultyController {
    final UserRepository users;
    final FeedbackRepository feedbacks;
    final ImprovementPlanRepository plans;
    final FeedbackAnalysisService ai;
    final AuditLogRepository audit;
    final NotificationService notifications;
    final QuestionRepository questions;

```

```

    public FacultyController(
        UserRepository u, FeedbackRepository f,
        ImprovementPlanRepository p, FeedbackAnalysisService a,
        AuditLogRepository l, NotificationService n, QuestionRepository q) {
        users = u;
        feedbacks = f;
        plans = p;
        ai = a;
        audit = l;
        notifications = n;
        questions = q;
    }
    User me(org.springframework.security.core.Authentication a) {
        return users.findByEmailIgnoreCase(a.getName()).orElseThrow();
    }
    @GetMapping("/dashboard") String dash(
        Model m, org.springframework.security.core.Authentication a) {
        User u = me(a);
        m.addAttribute("user", u);
        m.addAttribute(
            "feedbacks", feedbacks.findByFacultyIdOrderByCreatedAtDesc(u.getId())
        );
        m.addAttribute(
            "plans", plans.findByFacultyIdOrderByUpdatedAtDesc(u.getId())
        );
        m.addAttribute(
            "questions", questions.findByActiveTrueOrderByDisplayOrderAsc()
        );
        m.addAttribute(
            "avg", feedbacks.findByFacultyIdOrderByCreatedAtDesc(u.getId())
                .mapToInt(Feedback::getOverallRating)
                .stream()
                .orElse(0)
                .average()
        );
        return "faculty/dashboard";
    }
    @GetMapping("/plans/new") String newPlan(Model m) {
        m.addAttribute("form", new ImprovementPlanRequest());
        return "faculty/plan-form";
    }
    @PostMapping("/plans") String save(
    @Valid
    @ModelAttribute("form") ImprovementPlanRequest f,
    org.springframework.security.core.Authentication a) {
        User u = me(a);
        ImprovementPlan p = new ImprovementPlan();
        p.setFaculty(u);
        p.setTitle(f.getTitle());
        p.setProblemStatement(f.getProblemStatement());
        p.setEvidence(f.getEvidence());
        p.setRootCause(f.getRootCause());
        p.setProposedAction(f.getProposedAction());
        p.setTargetDate(f.getTargetDate());
        p.setPriority(f.getPriority());
        p.setExpectedOutcome(f.getExpectedOutcome());
        p.setMeasurementMethod(f.getMeasurementMethod());
        plans.save(p);
        return "redirect:/faculty/dashboard";
    }
    @PostMapping("/plans/{id}/submit") String submit(
    @PathVariable Long id, org.springframework.security.core.Authentication a)
    {
        ImprovementPlan p = plans.findById(id).orElseThrow();
        if (!p.getFaculty().getEmail().equalsIgnoreCase(a.getName()))
        {
            throw new
org.springframework.security.access.AccessDeniedException(
                "Not your plan"
            );
        }
    }

```

```

    }
    p.setStatus(ImprovementPlan.Status.SUBMITTED);
    plans.save(p);

    notifications.send(
        users.findByRole(User.Role.HOD).stream()
            .findFirst()
            .orElse(null),
        "Improvement plan submitted.",
        "A faculty improvement plan is ready for review.",
        "PLAN"
    );
    return "redirect:/faculty/dashboard";
}

@PostMapping("/questions") String addQuestion(
    @RequestParam String questionText,
    @RequestParam String category,
    org.springframework.security.core.Authentication a) {
    Question q = new Question();
    q.setQuestionText(questionText);
    q.setCategory(category);
    q.setDisplayOrder((int) questions.count() + 1);
    questions.save(q);
    return "redirect:/faculty/dashboard";
}

@PostMapping("/questions/{id}/disable") String disableQuestion(
    @PathVariable Long id) {
    Question q = questions.findById(id).orElseThrow();
    q.setActive(false);
    questions.save(q);
    return "redirect:/faculty/dashboard";
}

@GetMapping("/insights") String insights(
    org.springframework.security.core.Authentication a) {
    var fs =
        feedbacks.findByFacultyIdOrderByCreatedAtDesc(
            me(a).getId()
        );
    m.addAttribute("feedbacks", fs);
    m.addAttribute(
        "analyses",
        fs.stream()
            .map(a::analyze)
            .toList()
    );
    return "faculty/insights";
}
}

```

HodController.java:

```

package com.afims.controller;

import com.afims.entity.*;
import com.afims.repository.*;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/hod") public class HodController {
    final ImprovementPlanRepository plans;
    final FeedbackRepository feedbacks;
    final UserRepository users;
    final com.afims.service.MotificationService notifications;

    public HodController(ImprovementPlanRepository p, FeedbackRepository f, UserRepository u, com.afims.service.MotificationService n) {
        plans=p;
        feedbacks=f;
        users=u;
        notifications=n;
    }

    @GetMapping("/dashboard") String dash(Model m) {
        m.addAttribute("plans", plans.findAll());

```

```

        m.addAttribute("facultyCount", users.countByRole(User.Role.FACULTY));
        m.addAttribute("studentCount", users.countByRole(User.Role.STUDENT));

        m.addAttribute("avg", feedbacks.averageRating() != null ? 0 : feedbacks.averageRating());
        return "hod/dashboard";
    }

    @PostMapping("/plans/{id}/review") String review(
        @PathVariable Long id,
        @RequestParam String decision,
        @RequestParam(required=false) String comments) {
        ImprovementPlan p = plans.findById(id).orElseThrow();
        p.setHodComments(comments);
        p.setStatus(ImprovementPlan.Status.valueOf(decision));
        plans.save(p);
        notifications.send(p.getFaculty(), "Improvement plan reviewed", "Your HOD has updated the status to " + p.getStatus() + " : " + "PLAN");
        return "redirect:/hod/dashboard";
    }
}

```

StudentController.java:

```

package com.afims.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import com.afims.dto.FeedbackRequest;
import com.afims.entity.Feedback;
import com.afims.entity.FeedbackCycle;
import com.afims.entity.User;
import com.afims.repository.FeedbackCycleRepository;
import com.afims.repository.FeedbackRepository;
import com.afims.repository.QuestionRepository;
import com.afims.repository.UserRepository;
import com.afims.service.AuditService;
import com.afims.service.MotificationService;
import jakarta.validation.Valid;

@Controller
@RequestMapping("/student") public class StudentController {
    private final UserRepository users;
    private final FeedbackRepository feedbacks;
    private final FeedbackCycleRepository cycles;
    private final QuestionRepository questions;
    private final AuditService audit;
    private final MotificationService notifications;

    public StudentController(
        UserRepository users,
        FeedbackRepository feedbacks,
        FeedbackCycleRepository cycles,
        QuestionRepository questions,
        AuditService audit,
        MotificationService notifications) {
        this.users = users;
        this.feedbacks = feedbacks;
        this.cycles = cycles;
        this.questions = questions;
        this.audit = audit;
        this.notifications = notifications;
    }

    // Get currently logged-in user
    private User me(
        org.springframework.security.core.Authentication

```

```

authentication) {
    return
    users.findByEmailIgnoreCase(authentication.getName()).orElseThrow();
}

// Student dashboard
@GetMapping("/dashboard") public String dashboard(Model model,
org.springframework.security.core.Authentication authentication) {
    User user = me(authentication);
    model.addAttribute("user", user);
    model.addAttribute("feedbacks",
feedbacks.findByStudentIdOrderByCreatedAtDesc(user.getId()));
    model.addAttribute("faculties",
users.findByRole(User.Role.FACULTY));
    model.addAttribute("activeCycle",
cycles.findFirstByStatusOrderByStartDateDesc(FeedbackCycle.Status.A
CTIVE).orElse(null));
    return "student/dashboard";
}

// New feedback form
@GetMapping("/feedback/new") public String feedbackForm(Model
model, org.springframework.security.core.Authentication authentication) {
    model.addAttribute("form", new FeedbackRequest());
    model.addAttribute("faculties",
users.findByRole(User.Role.FACULTY));
    model.addAttribute("questions",
questions.findByActiveTrueOrderByDisplayOrderAsc());
    model.addAttribute("activeCycle",
cycles.findFirstByStatusOrderByStartDateDesc(FeedbackCycle.Status.A
CTIVE).orElse(null));
    return "student/feedback";
}

// Submit feedback
@PostMapping("/feedback") public String submitFeedback(
@Valid
@ModelAttribute("form") FeedbackRequest form, BindingResult
bindingResult, org.springframework.security.core.Authentication authentication,
Model model) {
    // Validation errors
    if (bindingResult.hasErrors()) {
        model.addAttribute("faculties",
users.findByRole(User.Role.FACULTY));
        model.addAttribute("questions",
questions.findByActiveTrueOrderByDisplayOrderAsc());
        model.addAttribute("activeCycle",
cycles.findFirstByStatusOrderByStartDateDesc(FeedbackCycle.Statu
s.ACTIVE).orElse(null));
        return "student/feedback";
    }

    User student = me(authentication);
    // Find active feedback cycle
    FeedbackCycle cycle =
cycles.findFirstByStatusOrderByStartDateDesc(FeedbackCycle.Status.A
CTIVE).orElseThrow(() -> new
IllegalStateException("No active feedback
cycle."));
    // Prevent duplicate feedback
    if
(feedbacks.existsByStudentIdAndFacultyIdAndCourseAndCycleId(
student.getId(), form.getFacultyId(), form.getCourse(), cycle.getId())) {
        model.addAttribute("error", "Duplicate feedback is
not allowed for this faculty, course and active cycle.");
        model.addAttribute("faculties",
users.findByRole(User.Role.FACULTY));

```

```

        model.addAttribute("questions",
questions.findByActiveTrueOrderByDisplayOrderAsc());
        model.addAttribute("activeCycle", cycle);
        return "student/feedback";
    }

    // Create feedback
    Feedback feedback = new Feedback();
    feedback.setStudent(student);

    feedback.setFaculty(
users.findById(form.getFacultyId()).orElseThrow());
    feedback.setCourse(form.getCourse());
    feedback.setSubject(form.getSubject());
    feedback.setCycle(cycle);
    feedback.setOverallRating(form.getRating());
    feedback.setComment(form.getComment());
    feedback.setSuggestion(form.getSuggestion());
    feedback.setAnonymous(form.isAnonymous());
    feedbacks.save(feedback);

    // Audit log
    audit.log(student, "FEEDBACK_SUBMITTED",
"Feedback", feedback.getId(), null);

    // Notify faculty
    notifications.send(feedback.getFaculty(), "New feedback
received", "A student submitted feedback for " +
feedback.getCourse() + " " + "FEEDBACK");
    // Notify student
    notifications.send(student, "Feedback submitted",
"Your feedback was recorded successfully.", "FEEDBACK");
    return "redirect:/student/dashboard?submitted";
}
}

```

HomeController.java:

package com.afims.controller;

```

import com.afims.dto.RegisterRequest;
import com.afims.service.UserService;
import jakarta.validation.Valid;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

```

@Controller

public class HomeController {

private final UserService users;

```

public HomeController(UserService users) {
    this.users = users;
}

```

```

@GetMapping("/")
public String home() {
    return "home";
}

```

```

@GetMapping("/login")
public String login() {
    return "auth/login";
}

@GetMapping("/register")
public String register(Model model) {
    model.addAttribute("form", new RegisterRequest());
    return "auth/register";
}

@PostMapping("/register")
public String doRegister(
    @Valid @ModelAttribute("form") RegisterRequest form,
    BindingResult bindingResult,
    Model model) {

    if (bindingResult.hasErrors()) {
        return "auth/register";
    }

    try {
        users.register(form);
        model.addAttribute(
            "success",
            "Registration submitted. An administrator must activate the account before login."
        );
        return "auth/login";
    } catch (IllegalArgumentException e) {
        model.addAttribute("error", e.getMessage());
        return "auth/register";
    }
}

@GetMapping("/dashboard")
public String dashboard(Authentication authentication) {

    String role = authentication
        .getAuthorities()
        .stream()
        .findFirst()
        .map(grantedAuthority -> grantedAuthority.getAuthority())
        .orElse("");

    return switch (role) {
        case "ROLE_STUDENT" -> "redirect:/student/dashboard";
        case "ROLE_FACULTY" -> "redirect:/faculty/dashboard";
        case "ROLE_HOD" -> "redirect:/hod/dashboard";
        case "ROLE_ADMIN" -> "redirect:/admin/dashboard";
        default -> "redirect/";
    };
}
}

```

AuditService.java:
package com.afims.service;
import com.afims.entity.*;

```

import com.afims.repository.AuditLogRepository;
import org.springframework.stereotype.Service;

@Service public class AuditService {
    final AuditLogRepository repo;

    public AuditService(AuditLogRepository r) {
        repo=r;
    }

    public void log(User u,String a,String e,Long id,String ip) {
        repo.save(new AuditLog(u,a,e,id,ip));
    }
}

```

FeedbackAnalysisService.java:
package com.afims.service;
import com.afims.entity.Feedback;
import org.springframework.stereotype.Service;
import java.util.*;

```

@Service public class FeedbackAnalysisService {
    public Map<String, Object> analyze(Feedback f) {
        String text=((f.getComment()==null?"":f.getComment()))+"
"+(f.getSuggestion()==null?"":f.getSuggestion()).toLowerCase();
        List<String> improve=new ArrayList<>(),positive=new ArrayList<>();
        if(text.contains("practical")||text.contains("example")) improve.add("Practical examples");
        if(text.contains("doubt")||text.contains("clarif")) improve.add("Doubt clarification sessions");
        if(text.contains("communication")) improve.add("Communication");
        if(f.getOverallRating()>=4) positive.add("Strong overall learning experience");
        if(text.contains("clear")) positive.add("Teaching clarity");
        String sentiment=f.getOverallRating()>=4?"Positive":f.getOverallRating()<=2?"Needs attention":"Mixed";
        String
        priority=improve.size()>=2||f.getOverallRating()<=2?"High":improve.size()==1?"Medium":"Low";

        String rec=improve.isEmpty()?"Continue the current learning practices and monitor future feedback.":"Add practical demonstrations and structured doubt-clarification opportunities.";

        return
        Map.of("sentiment",sentiment,"improvementThemes",improve,"positiveThemes",positive,"priority",priority,"recommendation",rec);
    }
}

```

UserService.java:
package com.afims.service;
import com.afims.dto.RegisterRequest;
import com.afims.entity.User;
import com.afims.repository.UserRepository;
import org.springframework.stereotype.Service;

```

@Service
public class UserService {
    final UserRepository repo;

    public UserService(UserRepository r) {
        repo = r;
    }

    public User register(RegisterRequest u) {
        if (repo.findByEmailIgnoreCase(u.getEmail()).isPresent()) {

```

```

        throw new IllegalArgumentException("Email is already registered.");
    }

    if (!x.getPassword().equals(x.getConfirmPassword())) {
        throw new IllegalArgumentException("Passwords do not match.");
    }

    User u = new User();
    u.setFullName(x.getFullName());
    u.setEmail(x.getEmail().toLowerCase());
    u.setPhone(x.getPhone());
    // Academic/demo mode: store the entered password directly.
    u.setPassword(x.getPassword());
    u.setInstitution(x.getInstitution());
    u.setDepartment(x.getDepartment());
    u.setInstitutionId(x.getInstitutionId());
    u.setRole(x.getRole());
    u.setStatus(User.RegistrationStatus.PENDING);
    return repo.save(u);
}

public User byEmail(String e) {
    return repo.findByEmailIgnoreCase(e).orElseThrow();
}
}

```

UserRepository.java:

```

package com.afims.repository;
import com.afims.entity.User;
import java.util.*;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmailIgnoreCase(String email);
    long countByRole(User.Role role);
    long countByStatus(User.RegistrationStatus status);
    List<User> findByRole(User.Role role);
}

```

QuestionRepository.java:

```

package com.afims.repository;
import com.afims.entity.Question;
import java.util.*;
import org.springframework.data.jpa.repository.JpaRepository;

public interface QuestionRepository extends JpaRepository<Question, Long> {
    List<Question> findByActiveTrueOrderByDisplayOrderAsc();
}

```

ImprovementPlanRepository.java

```

package com.afims.repository;
import com.afims.entity.*;
import java.util.*;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ImprovementPlanRepository extends
JpaRepository<ImprovementPlan, Long> {
    List<ImprovementPlan> findByFacultyIdOrderByUpdatedAtDesc(Long id);
    long countByStatus(ImprovementPlan.Status status);
}

```

FeedbackRepository.java

```

package com.afims.repository;
import com.afims.entity.*;
import java.util.*;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;

public interface FeedbackRepository extends JpaRepository<Feedback, Long> {
    boolean existsByStudentIdAndFacultyIdAndCourseAndCycleId(Long s, Long f, String c, Long cy);
    List<Feedback> findByStudentIdOrderByCreatedAtDesc(Long id);
    List<Feedback> findByFacultyIdOrderByCreatedAtDesc(Long id);
    @Query("select avg(f.overallRating) from Feedback f") Double averageRating();
    @Query("select count(f) from Feedback f") long total();
    @Query("select count(f) from Feedback f where f.overallRating >= 4") long
    positive();
}

```

DepartmentRepository.java:

```

package com.afims.repository;
import com.afims.entity.*;
import org.springframework.data.jpa.repository.JpaRepository;

public interface DepartmentRepository extends JpaRepository<Department, Long> {
}

```

AuditLogRepository.java

```

package com.afims.repository;
import com.afims.entity.*;
import org.springframework.data.domain.*;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AuditLogRepository extends JpaRepository<AuditLog, Long> {
    Page<AuditLog> findAllByOrderByTimestampDesc(Pageable p);
}

```

User.java:

```

package com.afims.entity;
import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name="users", uniqueConstraints=
@UniqueConstraint(name="uk_users_email",columnNames="email"))public class User {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
    @Column(nullable=false) private String fullName;
    @Column(nullable=false) private String email;
    @Column(nullable=false) private String phone;
    @Column(nullable=false) private String password;
    @Enumerated(EnumType.STRING)
    @Column(nullable=false) private Role role;
    @Enumerated(EnumType.STRING)
    @Column(nullable=false) private RegistrationStatus
    status=RegistrationStatus.PENDING;
    private String institution;
    private String department;
    private String institutionId;
    @Column(nullable=false) private LocalDateTime createdAt=LocalDateTime.now();
    public enum Role {
        STUDENT,FACULTY,HOD,ADMIN
    }
    public enum RegistrationStatus {
        PENDING,ACTIVE,REJECTED,SUSPENDED
    }
}

```

```

public User() {
}
public Long getId() {
    return id;
}
public void setId(Long v) {
    id=v;
}
public String getFullName() {
    return fullName;
}
public void setFullName(String v) {
    fullName=v;
}
public String getEmail() {
    return email;
}
public void setEmail(String v) {
    email=v;
}
public String getPhone() {
    return phone;
}
public void setPhone(String v) {
    phone=v;
}
public String getPassword() {
    return password;
}
public void setPassword(String v) {
    password=v;
}
public Role getRole() {
    return role;
}
public void setRole(Role v) {
    role=v;
}
public RegistrationStatus getStatus() {
    return status;
}
public void setStatus(RegistrationStatus v) {
    status=v;
}
public String getInstitution() {
    return institution;
}
public void setInstitution(String v) {
    institution=v;
}
public String getDepartment() {
    return department;
}
public void setDepartment(String v) {
    department=v;
}
public String getInstitutionId() {
    return institutionId;
}

```

```

public void setInstitutionId(String v) {
    institutionId=v;
}
public LocalDateTime getCreatedAt() {
    return createdAt;
}
public void setCreatedAt(LocalDateTime v) {
    createdAt=v;
}
}

```

Feedback.java:

```

package com.afims.entity;
import jakarta.persistence.*;
import java.time.LocalDateTime;
@Entity
@Table(name="feedback",uniqueConstraints=
@UniqueConstraint(name="uk_feedback_duplicate".columnNames= {
    "student_id","faculty_id","course","cycle_id"
}
)) public class Feedback {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY) Long id:
    @ManyToOne(optional=false)
    @JoinColumn(name="student_id") User student:
    @ManyToOne(optional=false)
    @JoinColumn(name="faculty_id") User faculty:
    @Column(nullable=false) String course:
    String subject:
    @ManyToOne(optional=false)
    @JoinColumn(name="cycle_id") FeedbackCycle cycle:
    int overallRating:
    boolean anonymous:
    @Column(length=2000) String comment:
    @Column(length=1000) String suggestion:
    LocalDateTime createdAt=LocalDateTime.now();
    public Feedback() {
    }
    public Long getId() {
        return id;
    }
    public User getStudent() {
        return student;
    }
    public void setStudent(User v) {
        student=v;
    }
    public User getFaculty() {
        return faculty;
    }
    public void setFaculty(User v) {
        faculty=v;
    }
    public String getCourse() {
        return course;
    }
    public void setCourse(String v) {
        course=v;
    }
}

```

```

    public String getSubject() {
        return subject;
    }
    public void setSubject(String v) {
        subject=v;
    }
    public FeedbackCycle getCycle() {
        return cycle;
    }
    public void setCycle(FeedbackCycle v) {
        cycle=v;
    }
    public int getOverallRating() {
        return overallRating;
    }
    public void setOverallRating(int v) {
        overallRating=v;
    }
    public boolean isAnonymous() {
        return anonymous;
    }
    public void setAnonymous(boolean v) {
        anonymous=v;
    }
    public String getComment() {
        return comment;
    }
    public void setComment(String v) {
        comment=v;
    }
    public String getSuggestion() {
        return suggestion;
    }
    public void setSuggestion(String v) {
        suggestion=v;
    }
    public LocalDateTime getCreatedAt() {
        return createdAt;
    }
}

```

Question.java:

```

package com.afims.entity;
import jakarta.persistence.*;
import java.time.LocalDateTime;
@Entity
@Table(name="questions") public class Question {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY) Long id;
    @Column(nullable=false) String questionText;
    String category;
    String ratingType="RATING_1.5";
    boolean required=true,active=true;
    Integer displayOrder=1;
    String targetRole="STUDENT";

```

```

    LocalDateTime createdAt=LocalDateTime.now();
    public Question() {
    }
    public Long getId() {
        return id;
    }
    public String getQuestionText() {
        return questionText;
    }
    public void setQuestionText(String v) {
        questionText=v;
    }
    public String getCategory() {
        return category;
    }
    public void setCategory(String v) {
        category=v;
    }
    public String getRatingType() {
        return ratingType;
    }
    public void setRatingType(String v) {
        ratingType=v;
    }
    public boolean isRequired() {
        return required;
    }
    public void setRequired(boolean v) {
        required=v;
    }
    public boolean isActive() {
        return active;
    }
    public void setActive(boolean v) {
        active=v;
    }
    public Integer getDisplayOrder() {
        return displayOrder;
    }
    public void setDisplayOrder(Integer v) {
        displayOrder=v;
    }
    public String getTargetRole() {
        return targetRole;
    }
    public void setTargetRole(String v) {
        targetRole=v;
    }
    public LocalDateTime getCreatedAt() {
        return createdAt;
    }
}

```

B.7.3.Screenshots:

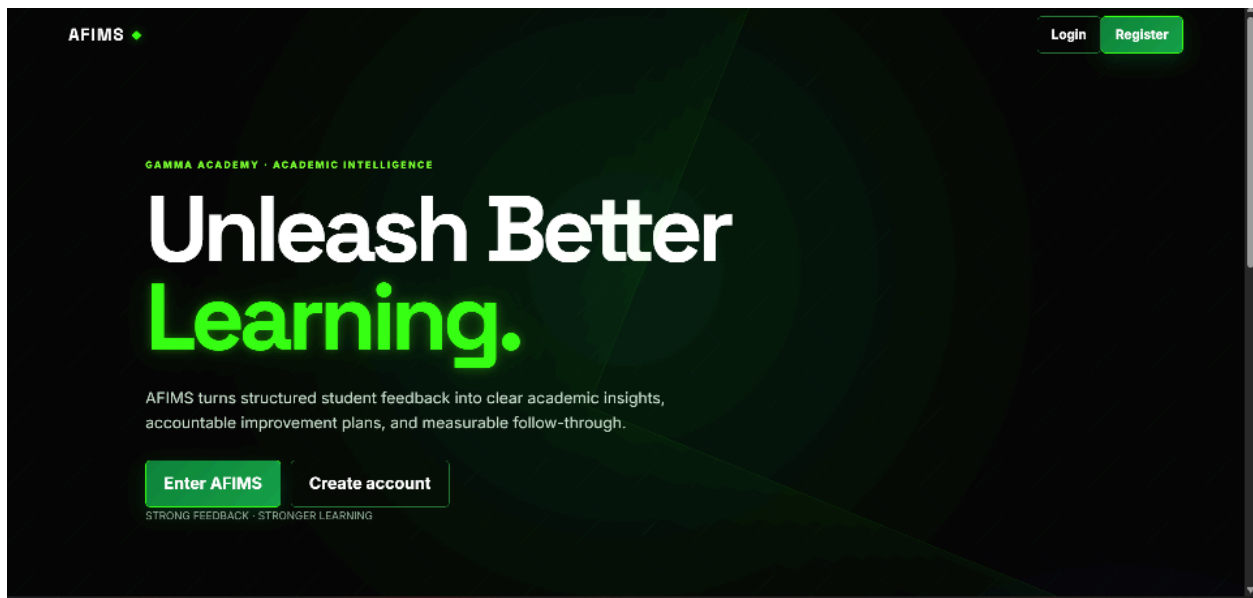


Fig:8.7.3.1.Home page

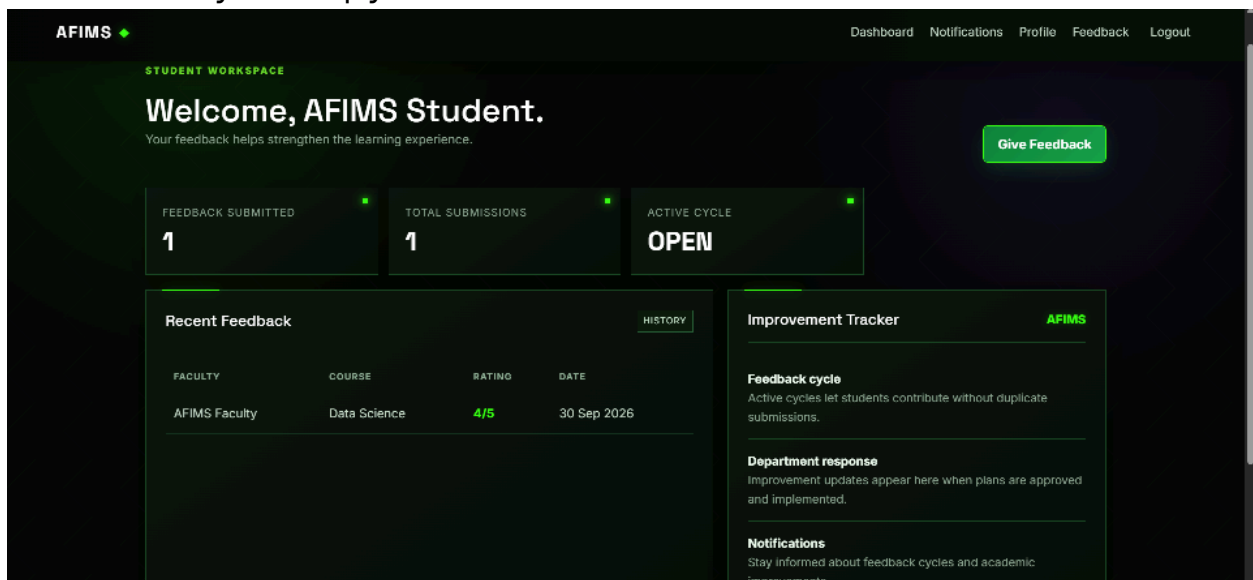


Fig:8.7.3.2.Student dashboard

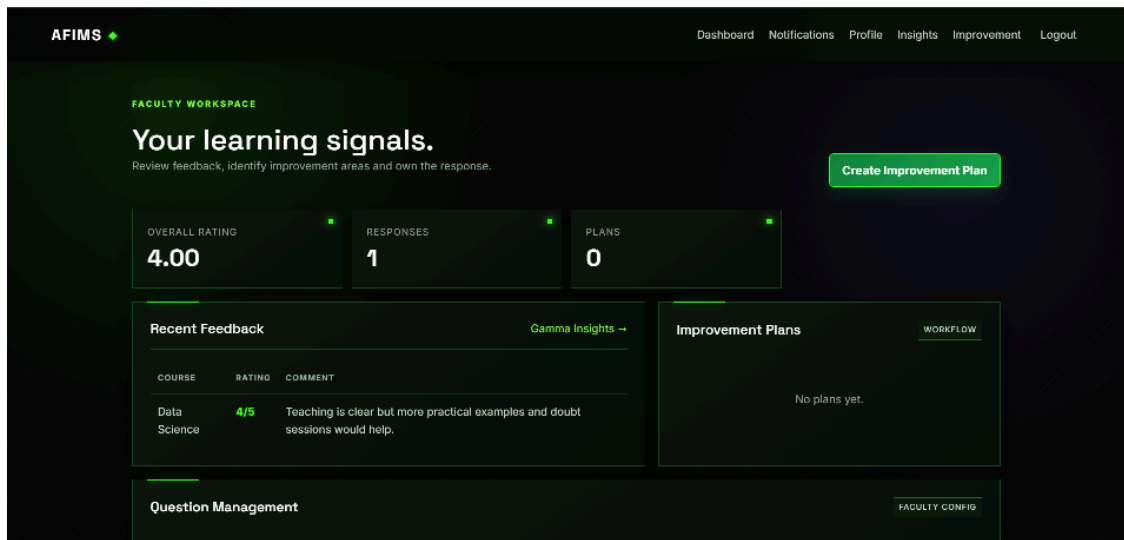


Fig:B.7.3.3.Faculty dashboard

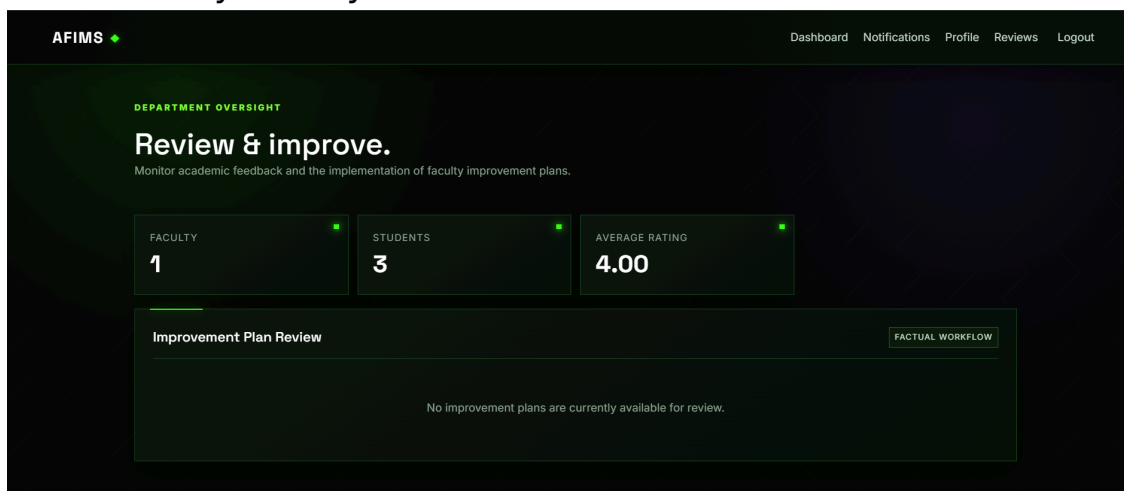


Fig:B.7.3.4.Hod dashboard

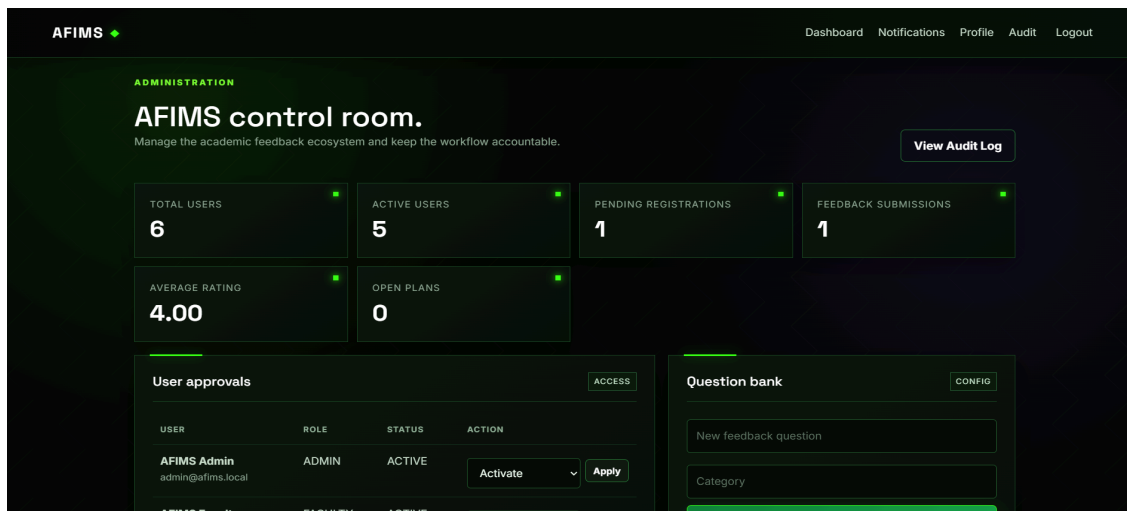


Fig:B.7.3.5.Admin dashboard

The "ACCOUNT REQUEST" form, titled "Join AFIMS," is used to create an institutional account. It includes the following fields and sections:

- Personal Information:**
 - Full Name: Text input field.
 - Email: Text input field with the value "fac1@gmail.com".
 - Phone: Text input field.
 - Role: Dropdown menu with the option "Select role".
- Institution Information:**
 - Institution: Text input field.
 - Department: Text input field.
 - Institution ID: Text input field.

A note at the top states: "Create an institutional account. New registrations remain pending until approved by an administrator."

Fig:B.7.3.6.Register page

The login page, titled "Welcome back," is for "SECURE ACADEMIC ACCESS." It includes the following elements:

- Sign in fields:**
 - Email: Text input field with the value "fac1@gmail.com".
 - Password: Text input field with masked characters "*****".
- Additional options:**
 - ☐ Remember me
 - [Forgot password?](#)
- Login button:** A red button labeled "Login to AFIMS".
- Registration link:** A link at the bottom that says "New to AFIMS? [Create an account](#)".

Fig:B.7.3.7.Login page

Appendix C: To Be Determined List

The following items are pending finalization and may be refined during the subsequent stages of system development and implementation.

No.	TBD Item	Description
1	Feedback Questionnaire Finalization	Finalization of the feedback questionnaire, including the number, wording, type, and categorization of questions.
2	Report Format Customization	Finalization of report formats, layouts, and the level of information to be included in generated reports.
3	Future Analytics Module	Detailed scope and functionality of an advanced analytics module, including additional analytical features and visualization requirements, to be determined based on future system requirements.

GitHub link:<https://github.com/BrisonOfficial/Academic-feedback-and-improvement-system>